

Cow book (3rd edition): Chapter 22

PARALLEL DBMS (PDBMS)

Finished overview of traditional RDBMS

Traditional DBMS

Structured data (relational), centralized, disk-based, OLTP workloads, ACID



PDBMS

Structured data (relational), parallel, disk-based, OLAP/MIXED workloads, ACID



Motivation

- Data is too large for single-disk solutions
- Parallel machines and multi-cores: all processors have multiple cores (this trend will still increase in the future)

Loading, indexing, query evaluation using multiple CPUs / servers and disks in parallel !

CPU parallelism

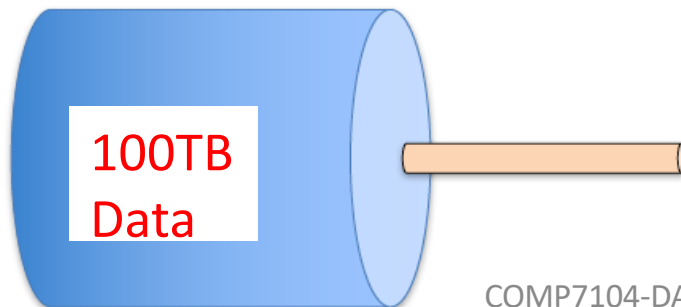
Sources of CPU parallelism:

- thread-level parallelism (based on Multiple Instruction Multiple Data – MIMD)
- data-level parallelism (based on Single Instruction Multiple Data – SIMD)

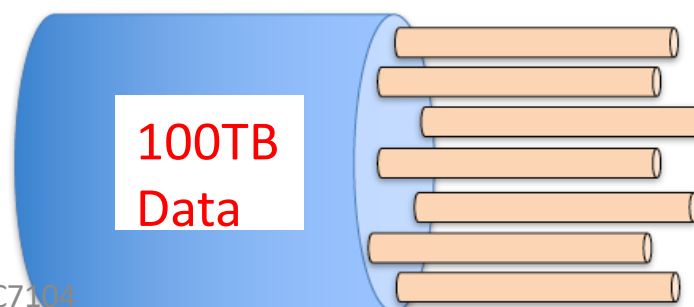
I/O parallelism

- Storage capacity has increased massively over the years, BUT **not access speeds**
- 1 disk to read 100TB (500 Mb/s) → 2.3 days !
 - even more to write
- Solution: multiple disks at once !
- 100 drives -> half an hour to read 100TB
 - In one machine? How big?
 - What about protection from hardware failure? MTBF ?
- One big data processing problem → many small data processing problems → Ideally: make them independent

At 500MB/s
2.3 days to scan



100 x parallel
30 minutes to scan



Observations on CPU + I/O parallelism

- Using more processes / threads to run queries in parallel will not help if disk is main bottleneck
- May decrease performance, if query is accessing different disk segments at the same time

Types of I/O parallelism

Split the DBMS across multiple storage devices to improve disk bandwidth latency

- Multiple disks per database
- One database per disk
- One relation per disk
- Split relation across multiple disks

Note: for durability too (RAID) – not our focus.

Easy for DBMSs to go parallel

- Lots of data:
 - Batch operations
 - Pre-existing divide-and-conquer algorithms
 - Natural pipelining
- Declarative languages & data transparency
 - Can adapt the parallelism strategy to the task and the hardware
 - All without changing the program!
 - Due to *physical data independence*

PDBMSs are the most successful application of parallelism!
Every major DBMS supports it.

Parallel DBMS

Database spread across multiple resources for

- large data sets (may not fit on one machine/node/disk)
- higher performance
- redundancy / fault-tolerance

Appears as a single logical database to the application, regardless of physical organization

→ SQL query for a centralized DBMS should give the same result on a parallel DBMS

Today: PDBMS (aka MPP – Massively Parallel Processing Systems)

Examples: Greenplum (EMC), Teradata, Netezza (IBM), Vertica (HP), ParAccel (Actian), DATAlegro (MS PDW), Amazon RedShift, Amazon Aurora, Cloudera Impala, etc

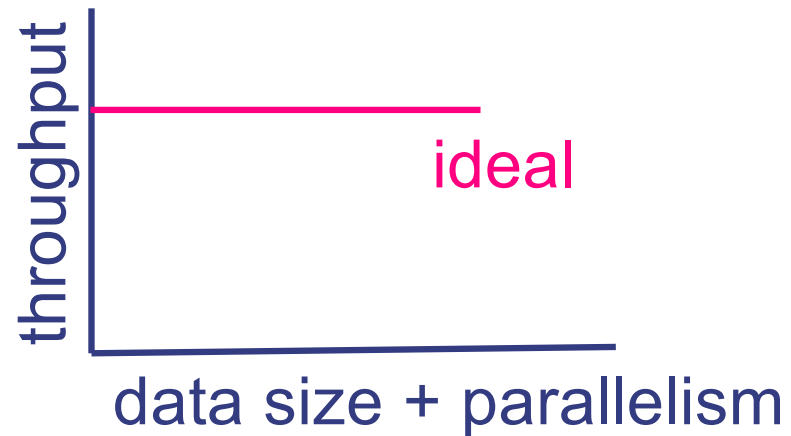
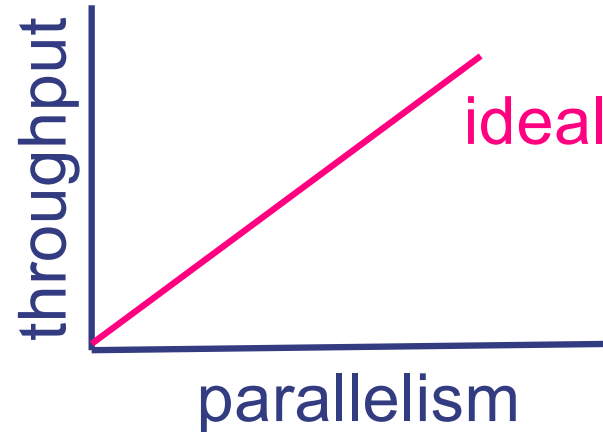
- Some may be deployed on high-end/high-performance, expensive, specialized hardware
- Declarative querying by design (SQL)
- Focus mostly on scalable solutions for **analytics**

Performance metrics for PDBMS

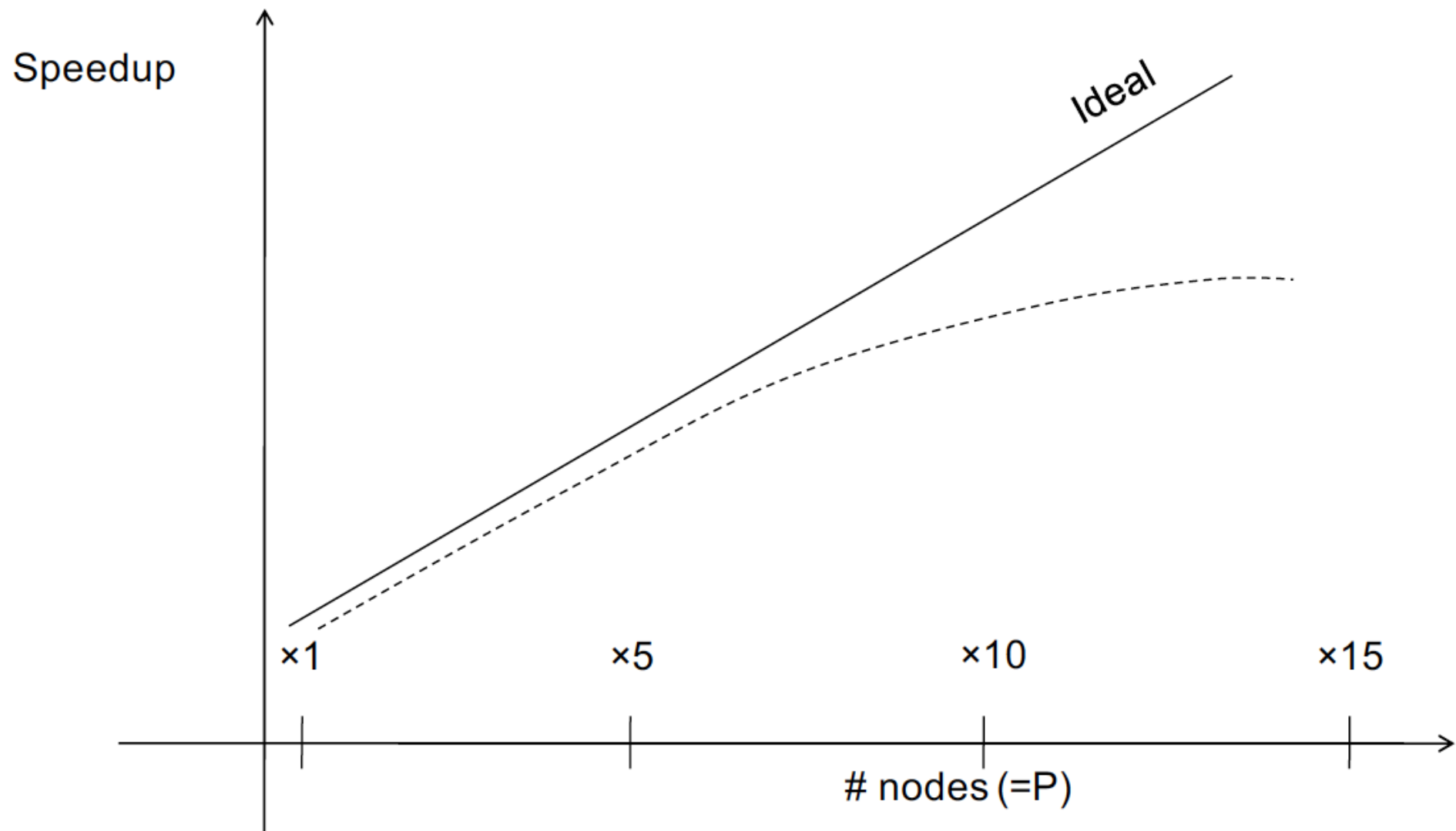
- Nodes = processors, computers
- Speedup: more nodes, same data at higher speed
- Scaleup: more nodes, more data at same speed
 - Also scale-out (more nodes) vs. scale-up (more computing power)
 - Horizontal scaling: more nodes → best cost / performance tradeoff
 - Vertical scaling: more computing power in same «box»
→ limited, diminishing returns

Two metrics to shoot for

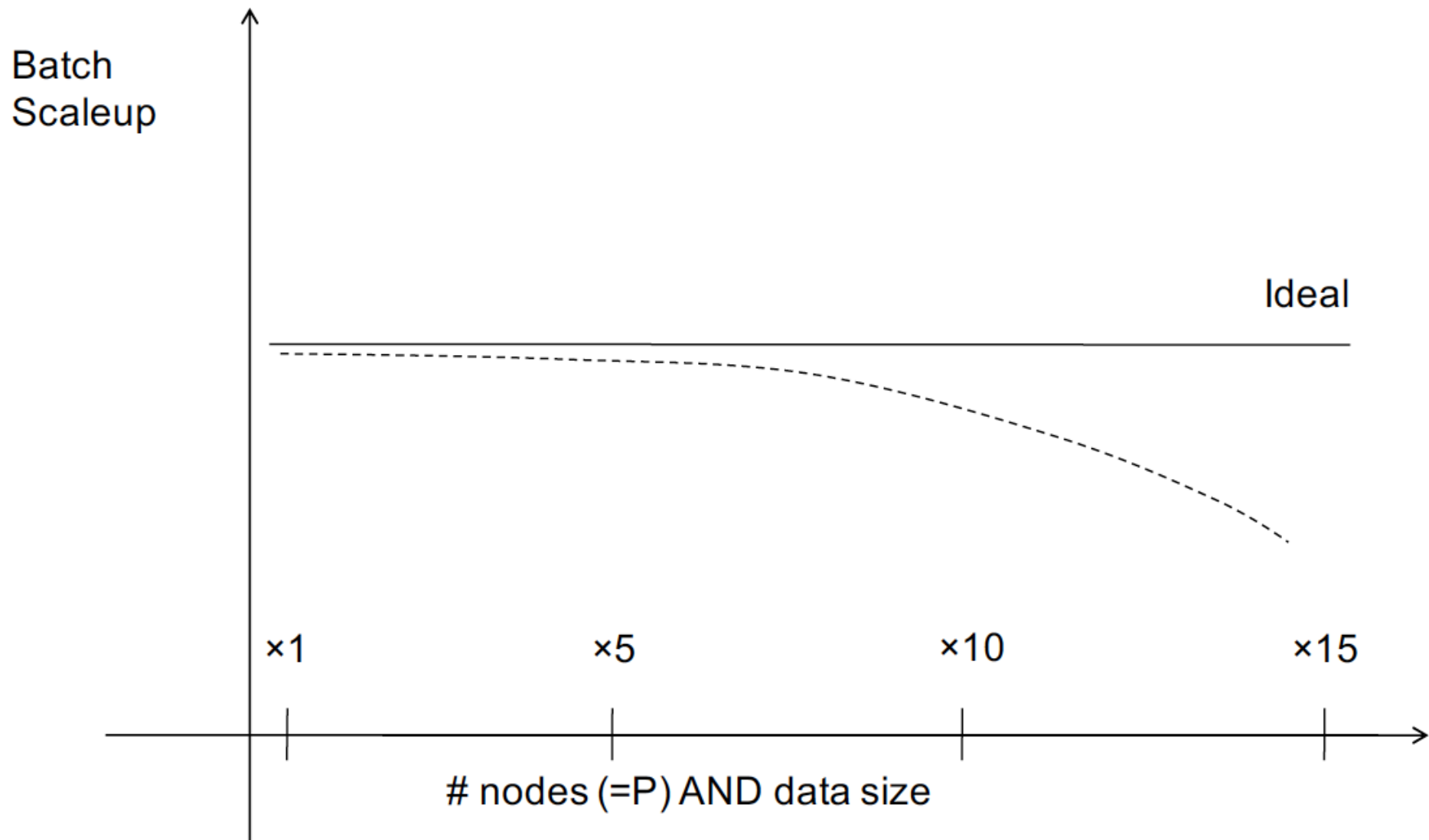
- Speed-up
 - Increase HW
 - Fix workload
- Scale-up
 - Increase HW
 - Increase workload



Linear v.s. Non-linear Speedup



Linear v.s. Non-linear Scaleup



Why sub-linear speedup and scaleup ?

- Startup cost
 - Cost of starting an operation on many nodes
- Interference
 - Contention for resources between nodes
- Skew
 - Slowest node becomes the bottleneck

Common architectures for PDBMS

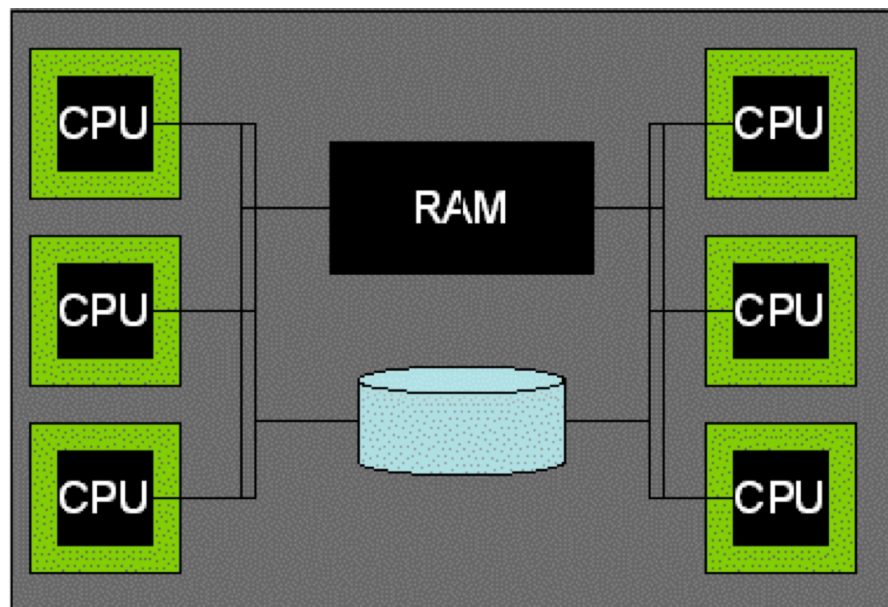
What shared resources are directly accessible to CPUs?

- Shared memory
- Shared disk
- Shared nothing

(shared-everything is what we discussed so far)

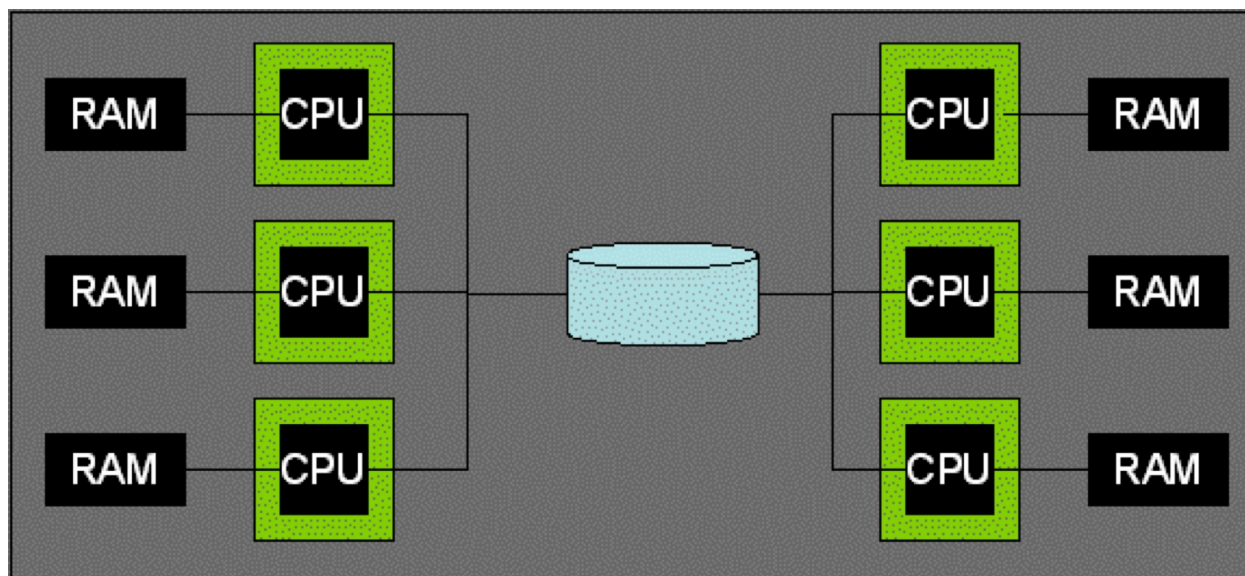
(next, nodes/CPU = DBMS instances)

Shared memory



- Nodes **share both RAM and disk**
- Dozens to hundreds of processors
- Example: SQL Server runs on a single machine and can leverage many threads to speed up a query
- Easy to use and program
- Expensive to scale, uncommon in PDBMSs today (few examples)
- But common in HPC architectures

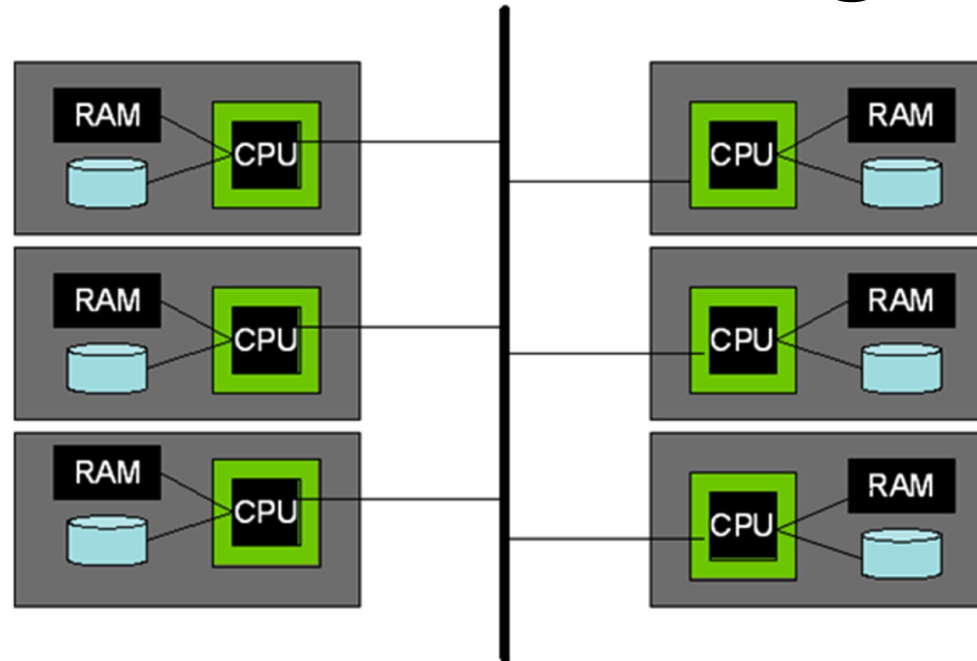
Shared disk



- Nodes access a single (logical) disk, are « stateless » (state is on disk)
- No worry about shared-memory, compute & storage layers can scale independently
 - Either found in largest "single-box" (non-cluster) multiprocessors (Oracle) → hard to scale (old)
 - Now in cloud-native PDBMSs (Snowflake, Google Spanner, Cloudera Impala, Amazon RedShift, Amazon Aurora – serverless & data lakes)

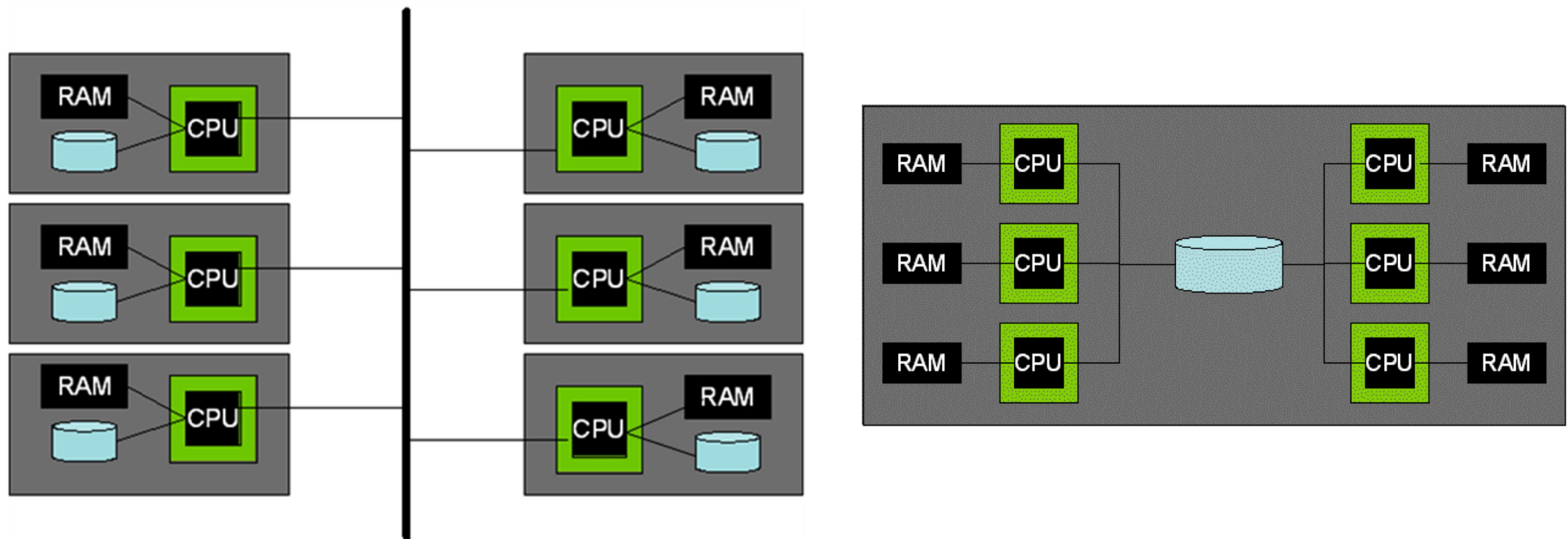


Shared nothing



- Cluster of commodity machines on high-speed network (made popular by NoSQL and Big Data; Cassandra, Teradata, MongoDB, Redis)
- Each node keeps state, has own memory and disk → lowest contention
- Easy to scale compute (high performance and efficiency)
- Harder to scale capacity and guarantee consistency
- Most difficult to program, administer, and tune

Shared nothing / Shared disk



- Newer systems are blurring the separation lines between shared-nothing & shared-disk
 - Machines have many cores (& disks), shared-nothing systems typically run many «nodes » on a single physical machine
 - Nodes in cloud-native systems may also have local disks for caching, temporary files (spilling data to disk), etc
- Our discussion applies to both.

Recap on common architectures for PDBMS

- Shared memory – easiest SW-wise, costliest HW-wise, almost no one does it today
- Shared nothing – cheap, high performance, hard to implement, hard to scale capacity & maintain consistency (many NoSQL systems)
- Shared disk – middle ground, compute & storage can scale independently, pbs with concurrency control – today most cloud-native DBMSs

Parallel vs. distributed DBMS

PDBMS

- Parallelization of various operations
 - e.g. loading data, building indexes, evaluating queries
- Nodes physically close, connected by high-speed LAN (ignore communication costs)
- Data may or may not be distributed initially
- Distribution is governed by **performance** consideration

DDBMS

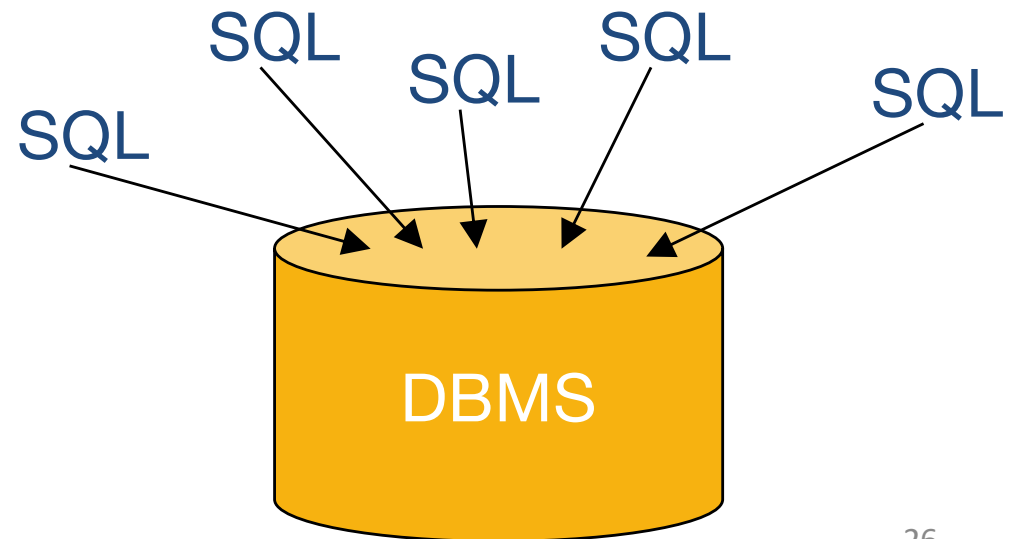
- Data is physically stored at different sites
 - each site is typically managed by an independent DBMS
- Location of data and autonomy of sites have an impact on query opt., CC and recovery
- Also governed by other factors:
 - increased availability for system crash
 - local ownership and access
- Nodes may even be adversarial

Will not discuss in this class.

PARALLEL QUERY PROCESSING

Kinds of query parallelism

- Inter-query (parallelism across queries)
 - Each query runs on a separate processor
 - Single thread (no parallelism) per query
 - Does require parallel-aware *Concurrency Control*
 - A topic we discussed already



Kinds of query parallelism

- Inter-query (parallelism across queries)
 - Each query runs on a separate processor
 - Single thread (no parallelism) per query
 - Does require parallel-aware *Concurrency Control*
 - A topic we discussed already
- Intra-query (within a single query)

Roughly 2 kinds of intra-query parallelism

: any sequential program,
e.g. a relational operator

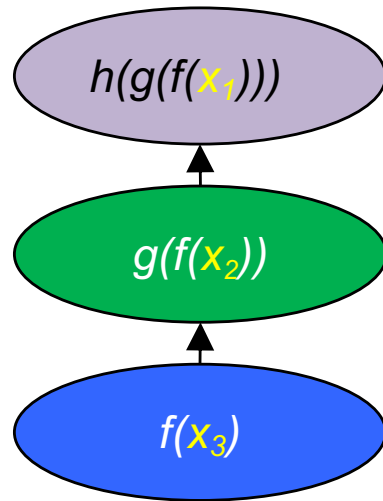
Legend

●: running on machine 1

●: running on machine 2

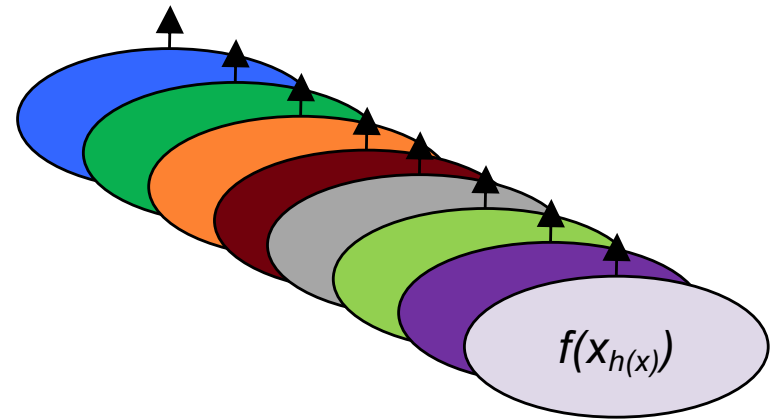
●: running on machine 3

...



Pipeline

scales up to pipeline depth



Partition (or data-partitioned)
scales up to amount of data

Both are natural in DBMS!

Inter-operator (pipeline) parallelism

Legend

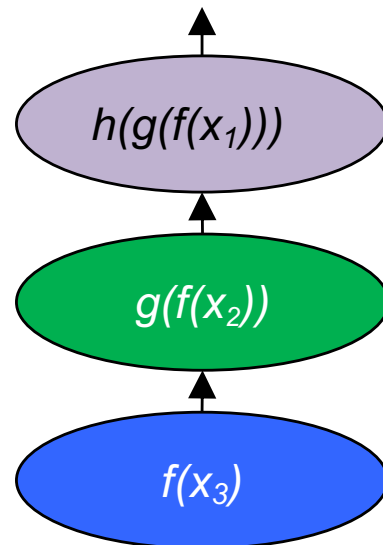
●: running on machine 1

●: running on machine 2

●: running on machine 3

- Intra-query

1. Inter-operator (between operators)



Pipeline parallelism

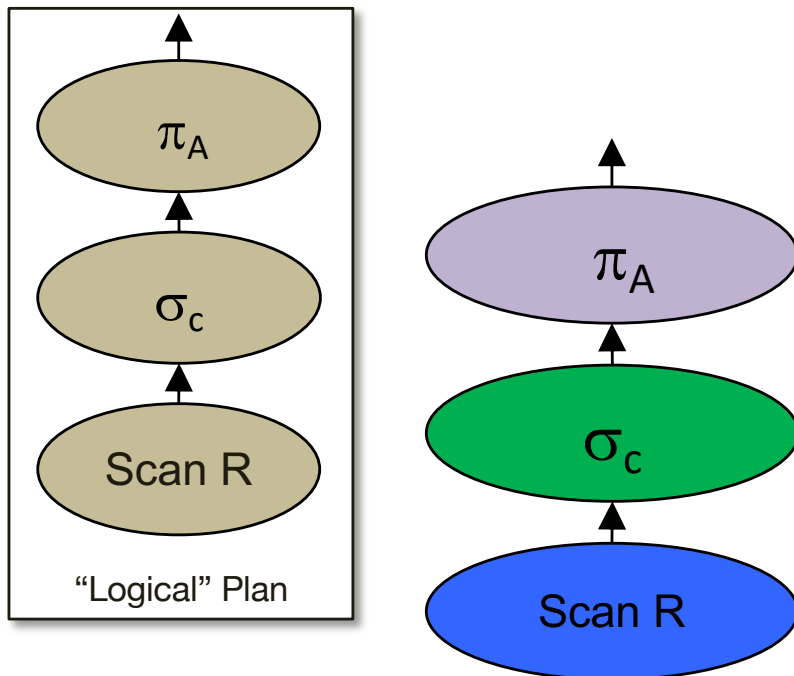
Inter-operator (pipeline) parallelism

Legend

- : running on machine 1
- : running on machine 2
- : running on machine 3

- Intra-query

1. Inter-operator (between operators)

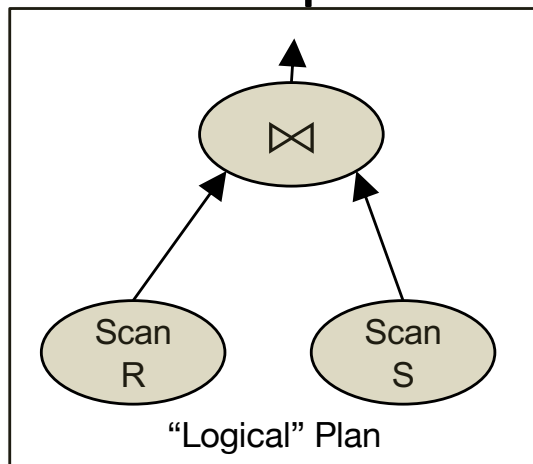


- Common in **streaming** systems (Kafka, Storm, Flink)
- Less common for OLAP
- Limited by pipeline-breakers

Intra-operator (partition) parallelism

- Intra-query

2. **Intra**-operator (within a single operator) – aka partition parallelism



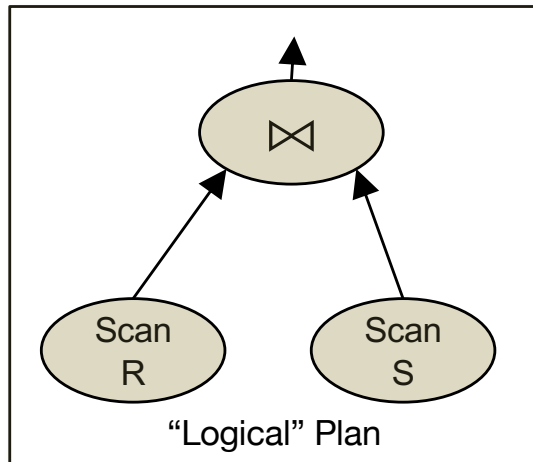
Intra-operator (partition) parallelism

Legend

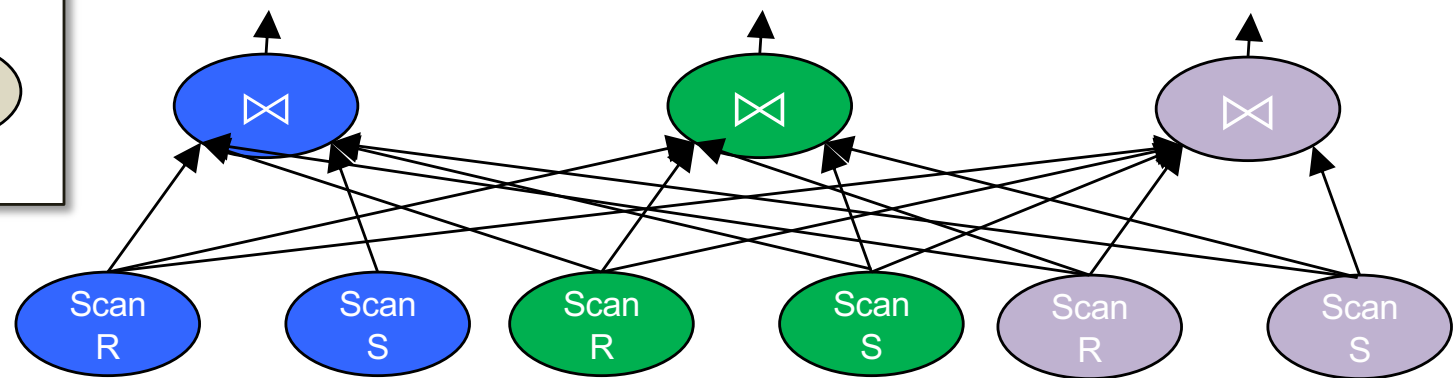
- : running on machine 1
- : running on machine 2
- : running on machine 3

- Intra-query

2. Intra-operator



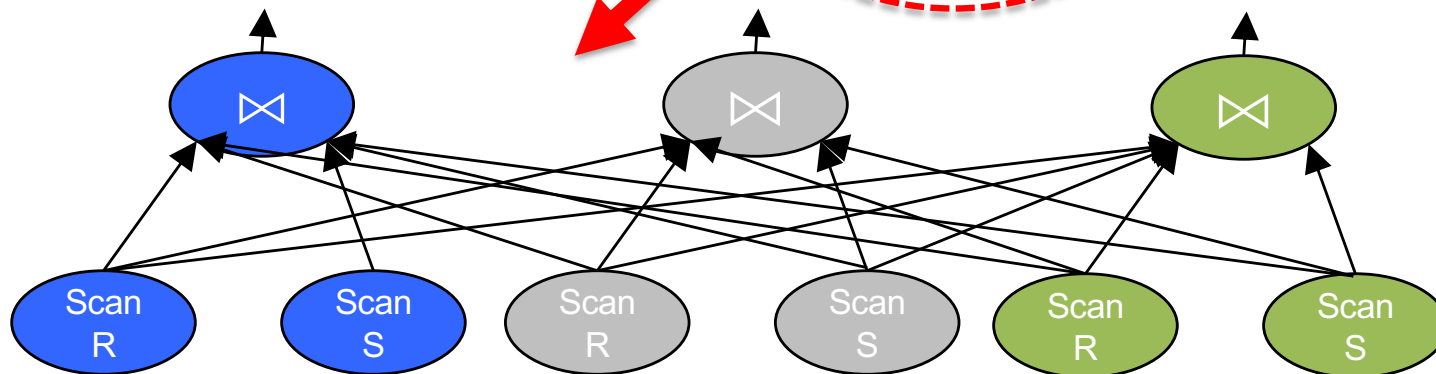
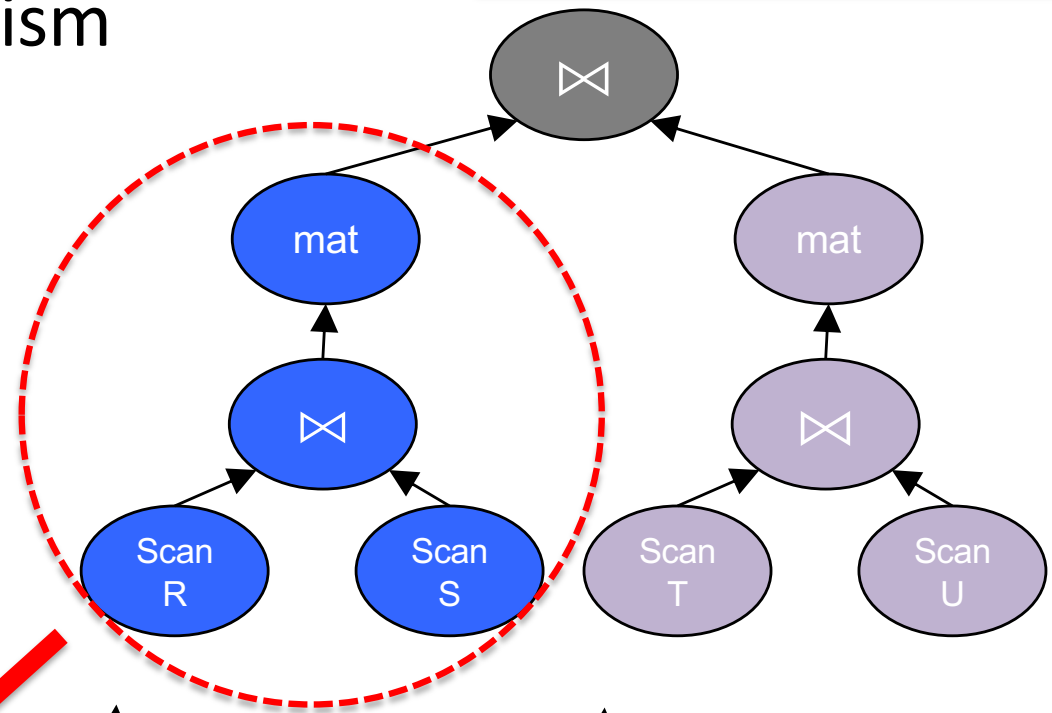
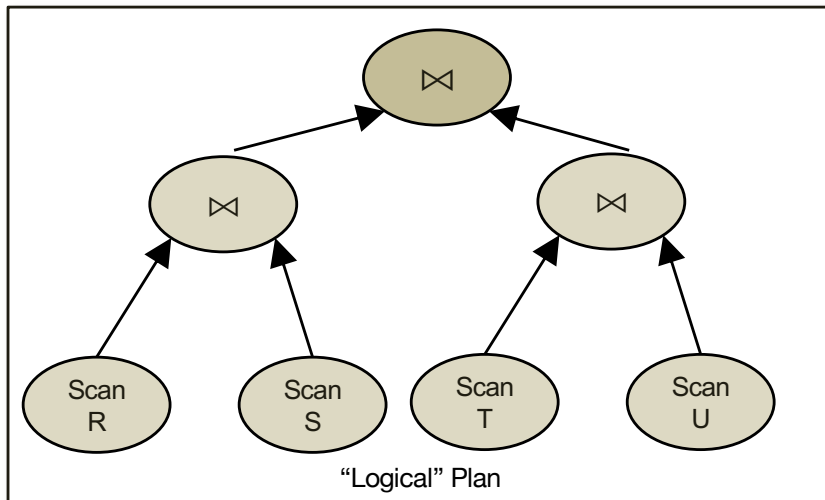
- Most common for OLAP



Hybrid intra-query parallelism

- Intra-query

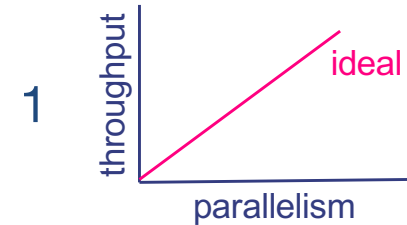
1. Bushy (tree) parallelism



Quick quizz

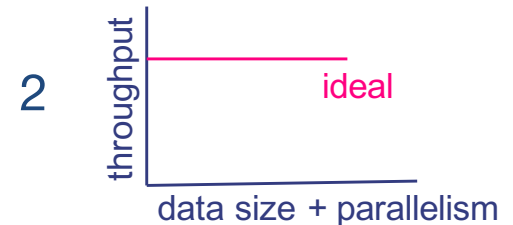
Match the metric to the ideal curve:

- A. Speed-up
- B. Scale-up



Which has more scaling potential

- A. Pipeline parallelism
- B. Partition parallelism



Which architectures allow all CPUs to access the same disk

- A. Shared memory
- B. Shared disk
- B. Shared nothing
- D. none of the above

Quick quizz

Match the term and definition:

- A. Intra-operator
- B. Inter-operator

- 1. Within an operator
- 2. Across operators

Match the general category with the kind of intra-query parallelism

- A. Intra-operator
- B. Inter-operator

- 1. Partition parallel
- 2. Bushy parallel
- 3. Pipeline parallel

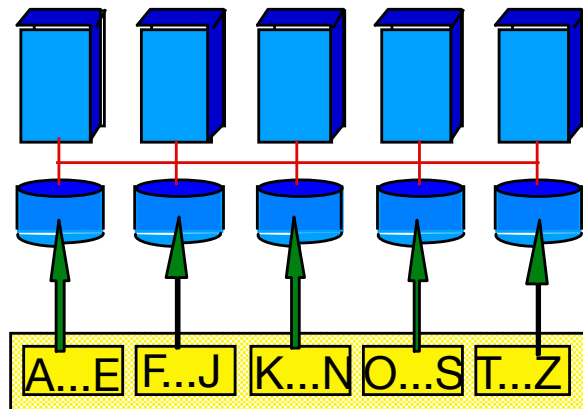
Scans and Selections

INTRA-OPERATOR PARALLELISM

Data partitioning (for I/O parallelism)

How to partition horizontally a table across disks / machines

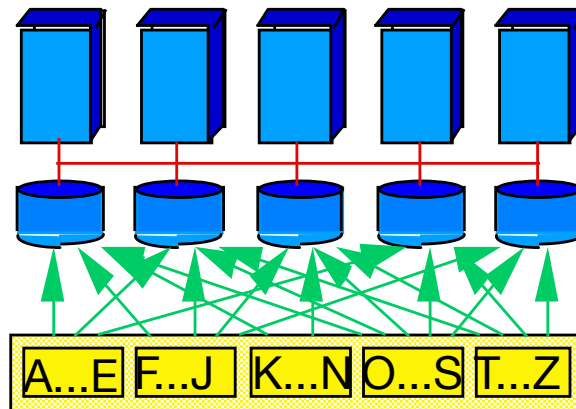
Range (e.g., by PK)



Good for equijoins,
range queries, group-by

- can lead to data skew

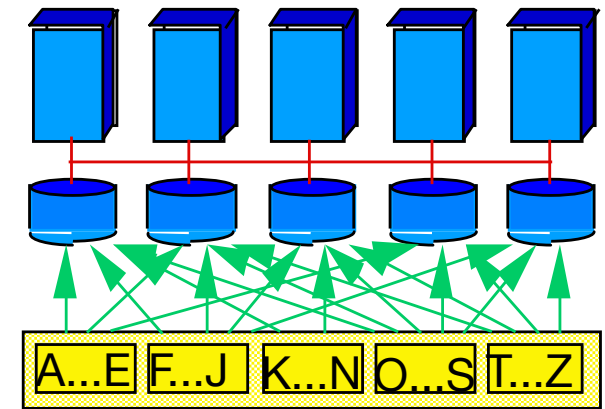
Hash (e.g. Cassandra)



Good for equijoins,
group-by, but only if
hashed on that attribute

- can lead to data skew

Round Robin



Good for spreading load,
full scans

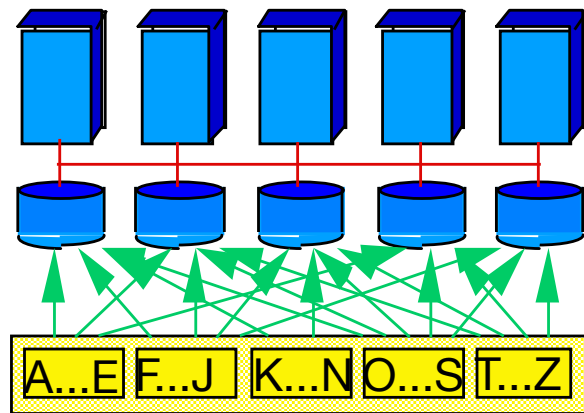
Shared nothing very sensitive to "good" partitioning !

Parallel scans & selections

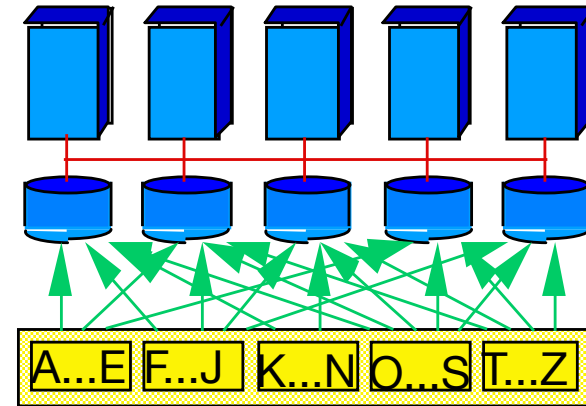
- Scan in parallel, merge (concatenate) output
- σ_p : skip sites that have no tuples satisfying p
 - range or hash partitioning
- σ_p : indexes can reduce lookup cost at each partition
- Question: How do indexes differ in the different data partitioning schemes?
 - Answer: In no way. Each partition has its own index that indexes whatever data is there.

Lookup by key

- Data partitioned on function of key (as in hash)?
- Otherwise



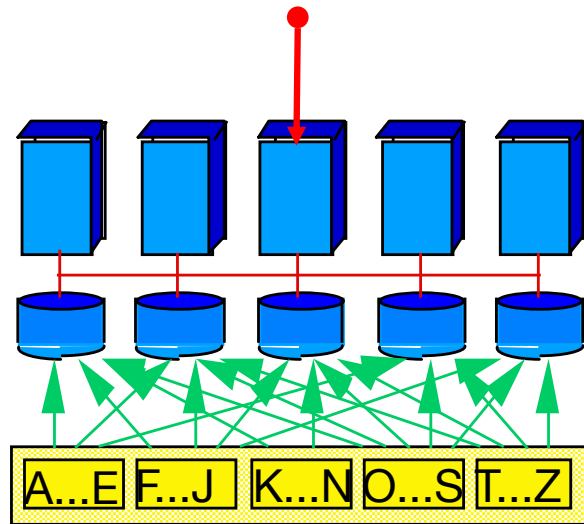
Hash partitioned



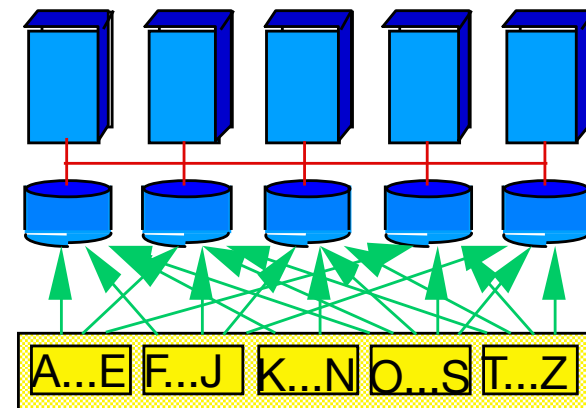
Round robin

Lookup by key

- Data partitioned on function of key (as in hash)?
 - Great! Route lookup only to relevant node
- Otherwise



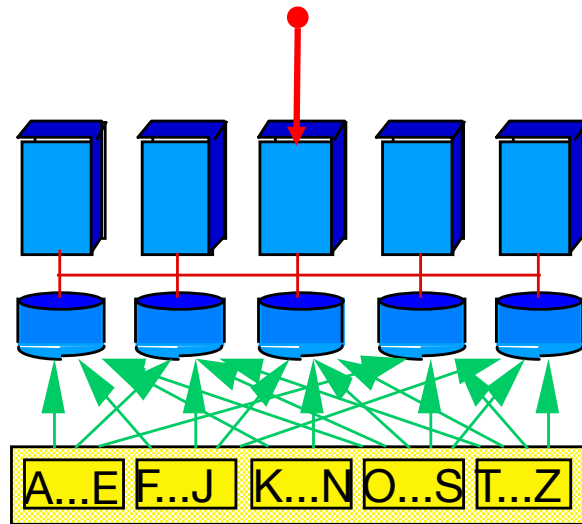
Hash partitioned



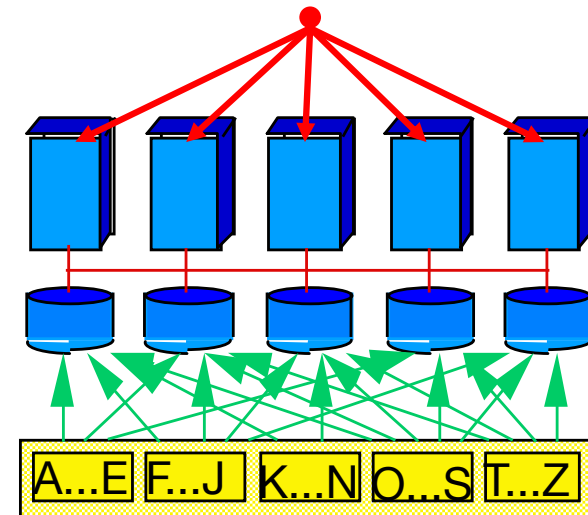
Round robin

Lookup by key

- Data partitioned on function of key (as in hash)?
 - Great! Route lookup only to relevant node
- Otherwise
 - Have to broadcast lookup (to all nodes)



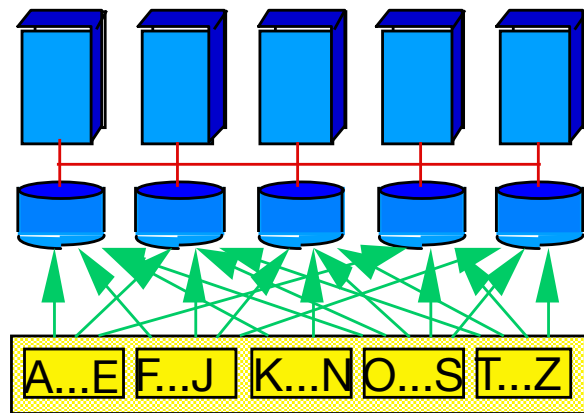
Hash partitioned



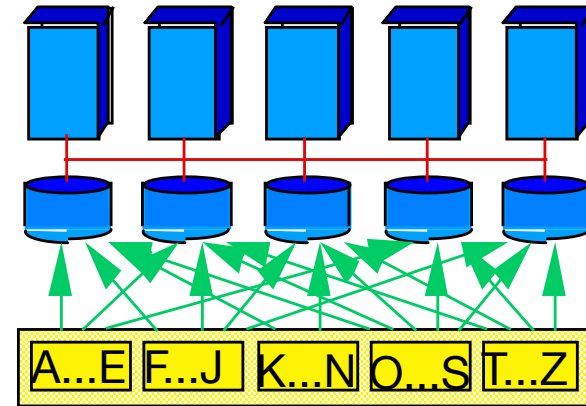
Round robin

What about insert?

- Data partitioned on function of key (as in hash)?
- Otherwise



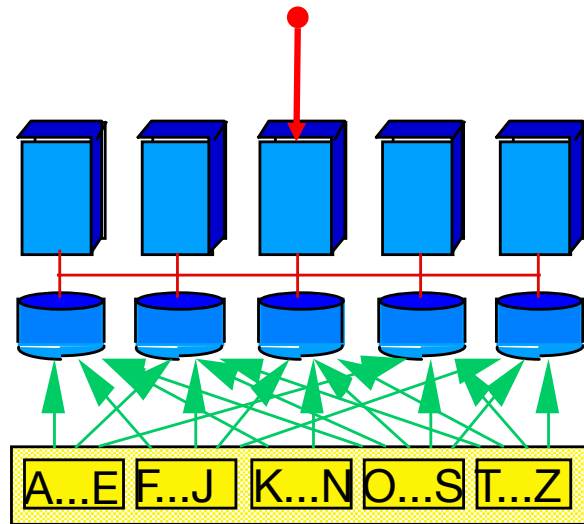
Hash partitioned



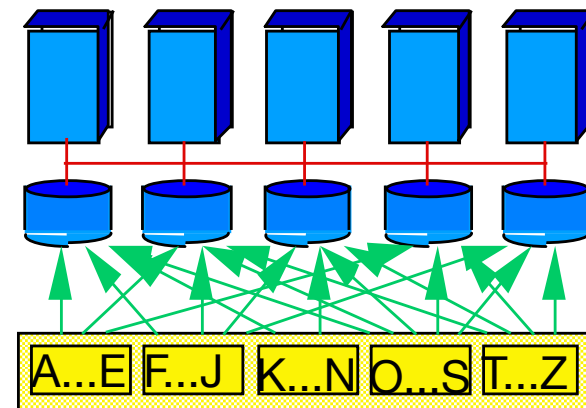
Round robin

What about insert?

- Data partitioned on function of key (as in hash)?
 - Route insert to relevant node
- Otherwise



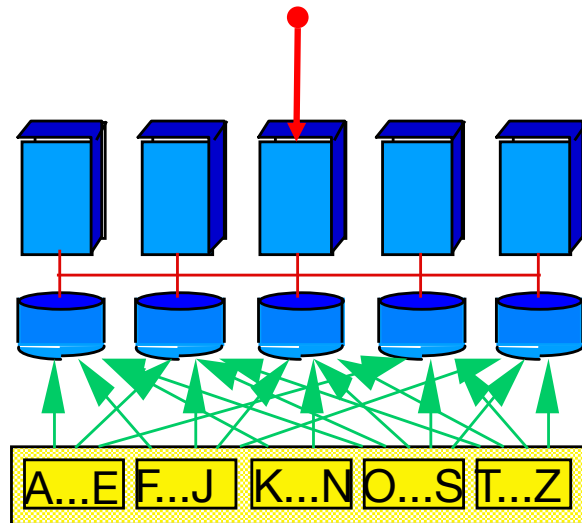
Hash partitioned



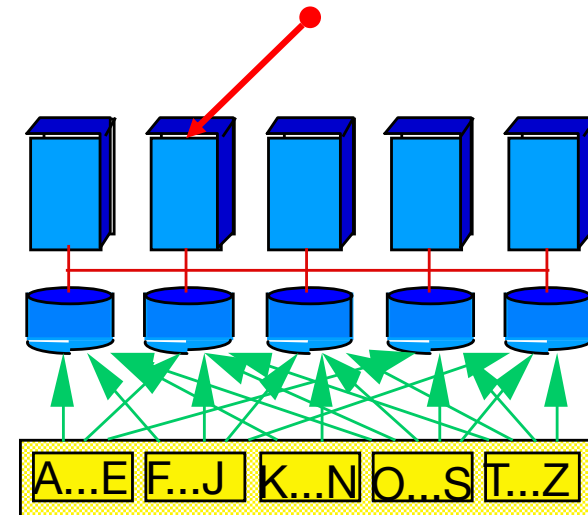
Round robin

What about insert?

- Data partitioned on function of key (as in hash)?
 - Route insert to relevant node
- Otherwise
 - Route insert to *any* node



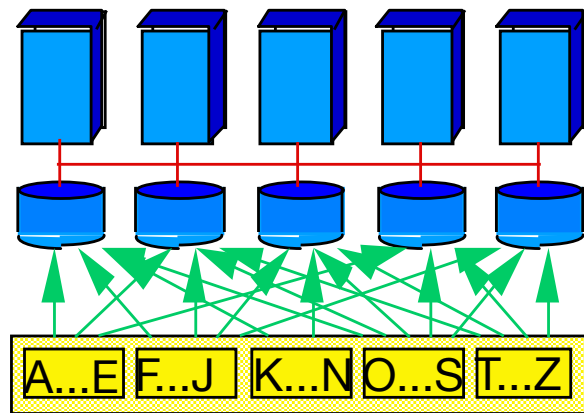
Hash partitioned



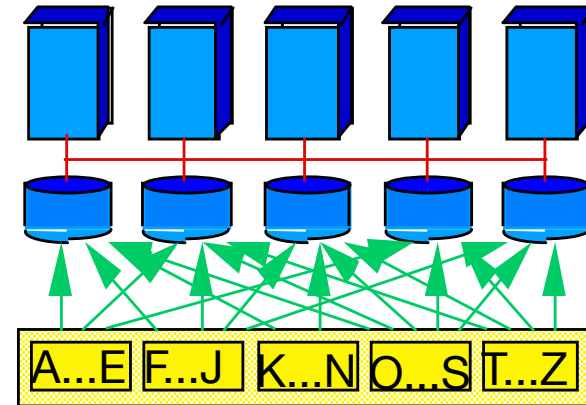
Round robin

Insert to unique key?

- Data partitioned on function of key (as in hash)?
- Otherwise



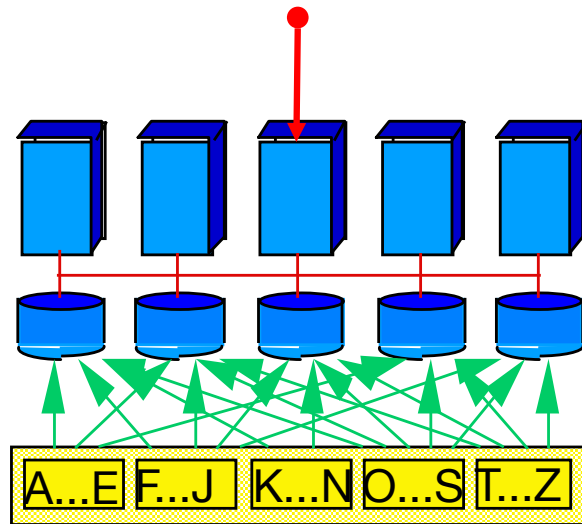
Hash partitioned



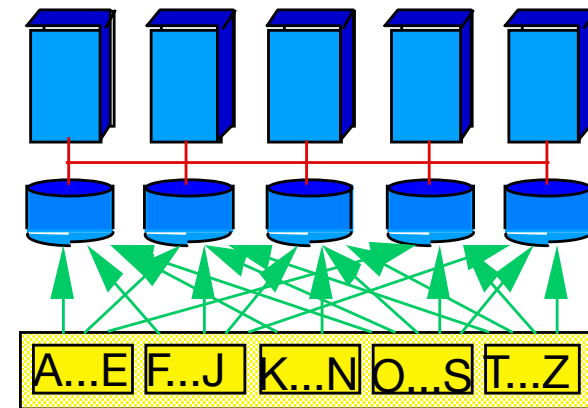
Round robin

Insert to unique key?

- Data partitioned on function of key (as in hash)?
 - Route to relevant node
 - And reject if already exists



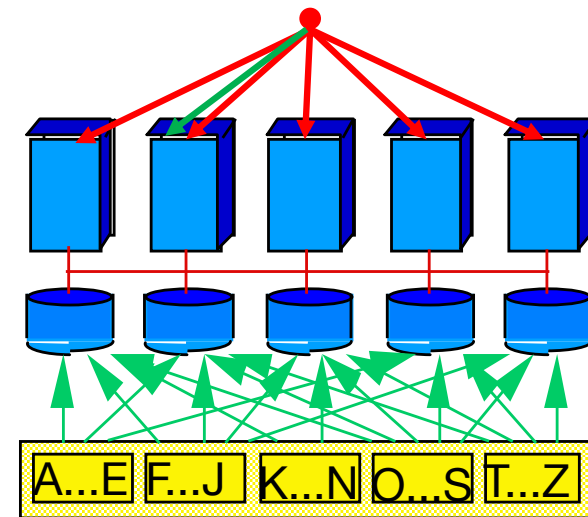
Hash partitioned



Round robin

Insert to unique key?

- Data partitioned on function of key (as in hash)?
 - Route to relevant node
 - And reject if already exists
- Otherwise
 - Broadcast lookup
 - Collect responses
 - If not exists, insert anywhere
 - Else reject



Round robin

Quick quizz

Which of the following partitioning schemes works well for equality queries?

- A. Range
- B. Hash
- C. Round Robin

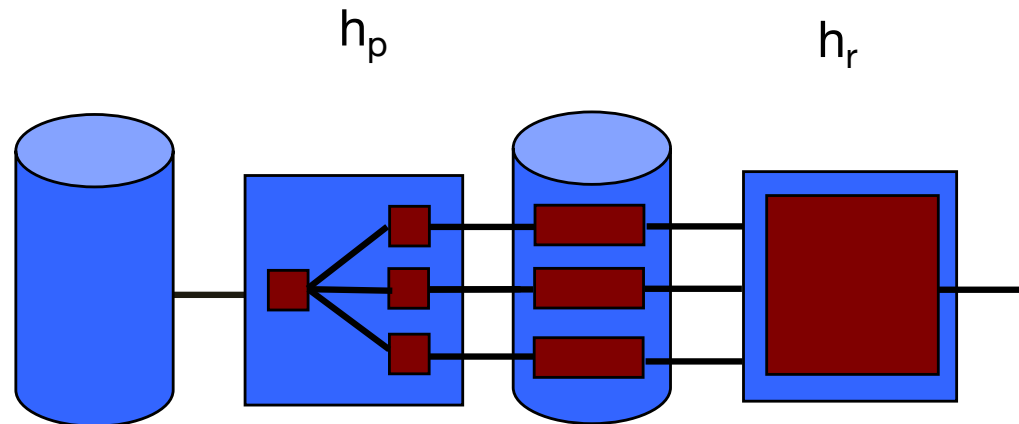
Inserting to a key with a uniqueness constraint requires:

- A. Doing a lookup, followed by an insert if lookup is empty
- B. Contacting a single node when hash partitioned by the key
- C. Broadcasting to all nodes when round-robin partitioned
- D. All of the above

Hashing

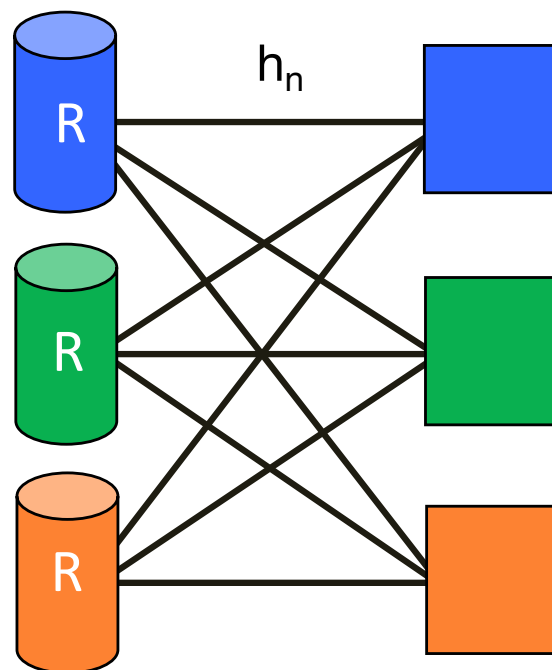
INTRA-OPERATOR PARALLELISM

Remember hashing?



Parallelize me! Hashing

- Phase 1: *shuffle* data across machines (h_n)
 - streaming out to network as it is scanned
 - which machine for this record?
use (yet another) independent hash function h_n

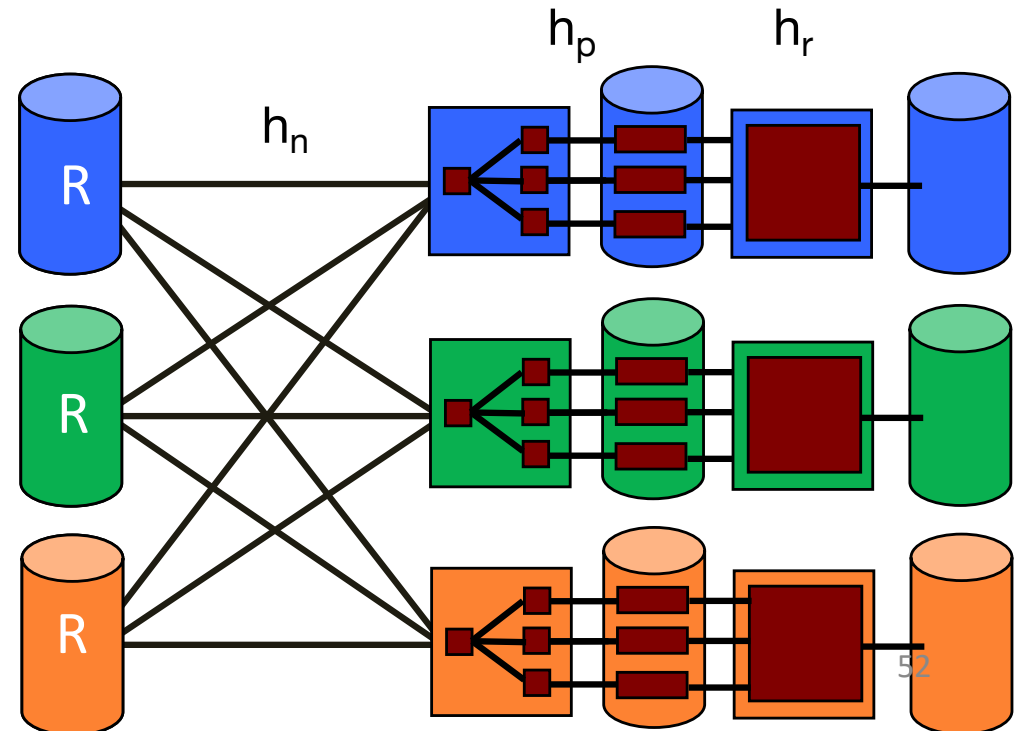


Parallelize me! Hashing

- Phase 1: *shuffle* data across machines (h_n)
 - streaming out to network as it is scanned
 - which machine for this record?
use (yet another) independent hash function h_n
- Receivers proceed with phase 1 in a pipeline *as data streams in*
 - from local disk and network

So easy!
Nearly same as single-node hashing

Near-perfect speed-up, scale-up!
Every component works at its top
speed, no waiting



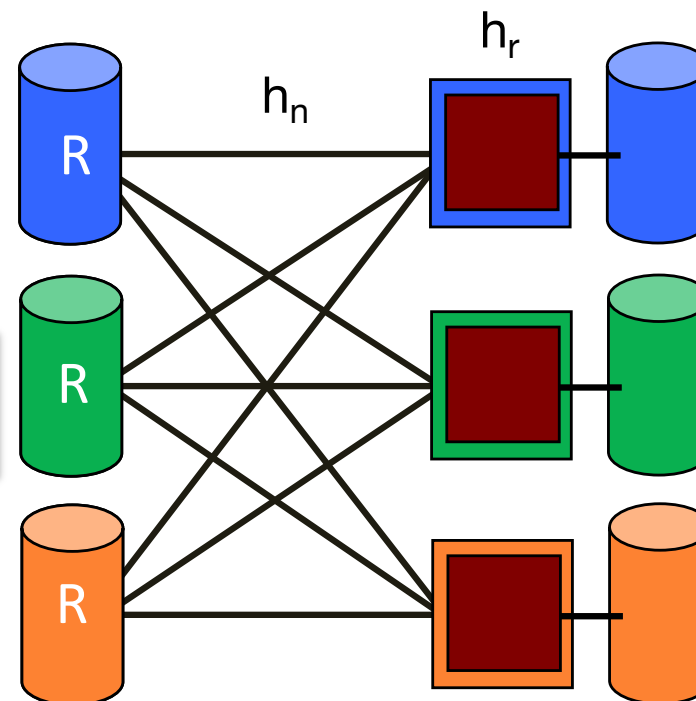
Hash join ? If you have enough machines...

Naïve parallel hash join

- Phase 1: *shuffle* data across machines (h_n)
 - Parallel scan streaming out to network
 - Which machine for a given record?
Use (yet another) independent hash function h_n
- Receivers proceed with naïve hashing in a pipeline *as data streams in*
 - from local disk and network

Smaller relation R fits across machines ?

Parallel hashing of larger relation



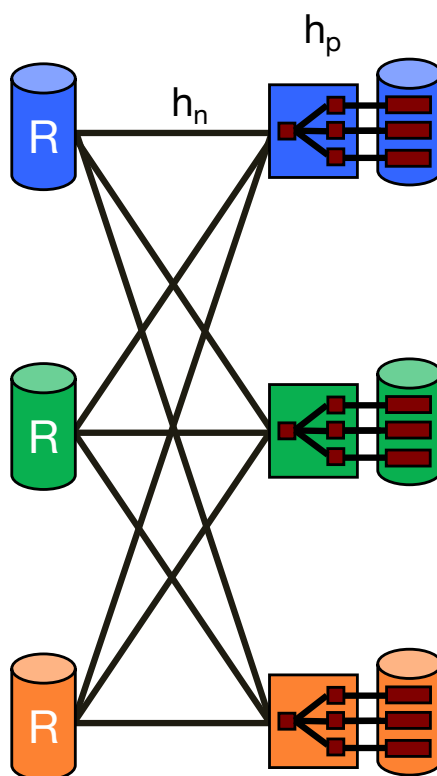
So easy!
Nearly same as single-node naïve hash join

Near-perfect speed-up, scale-up!
Every component works at its top speed, no waiting

Parallel Grace hash join

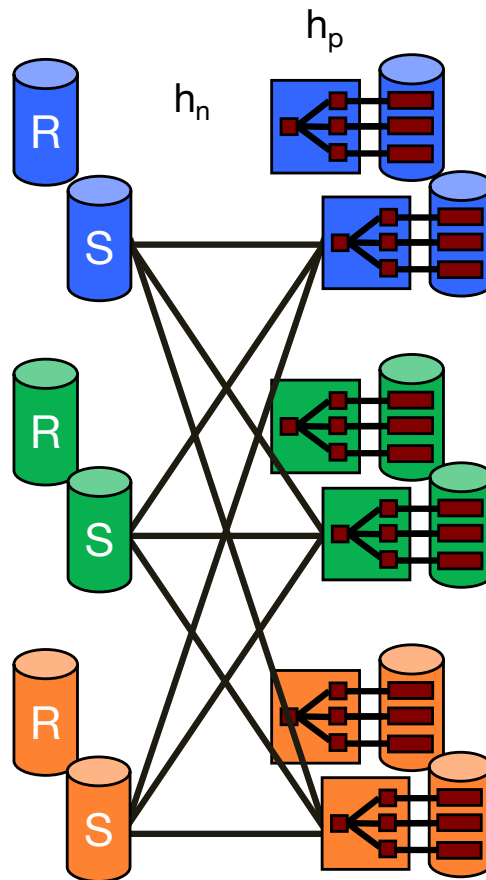
Motivation: if the joining relations are very large, and the number of h_n partitions is chosen equal to the number of processors, the size of each partition may be large $\rightarrow$ high cost for each local join

- Pass 1 is like hashing above



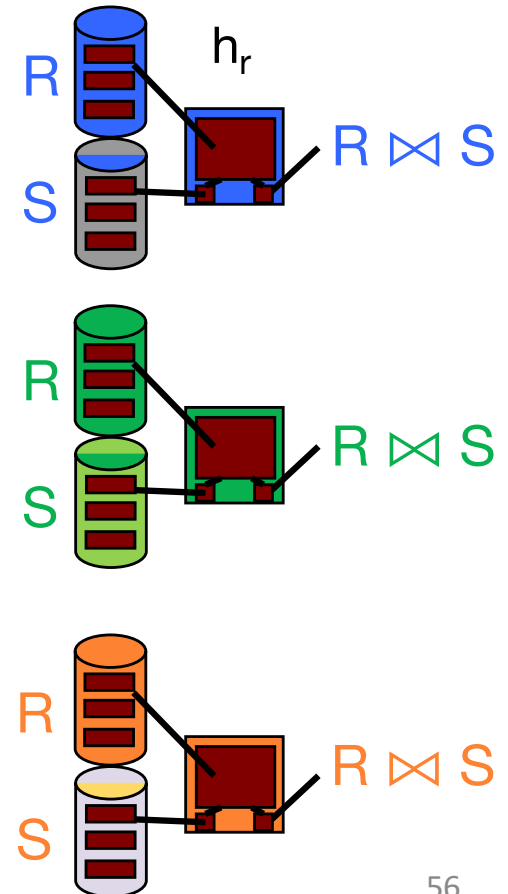
Parallel Grace hash join

- Pass 1 is like hashing above
 - But do it 2x: once for each relation being joined



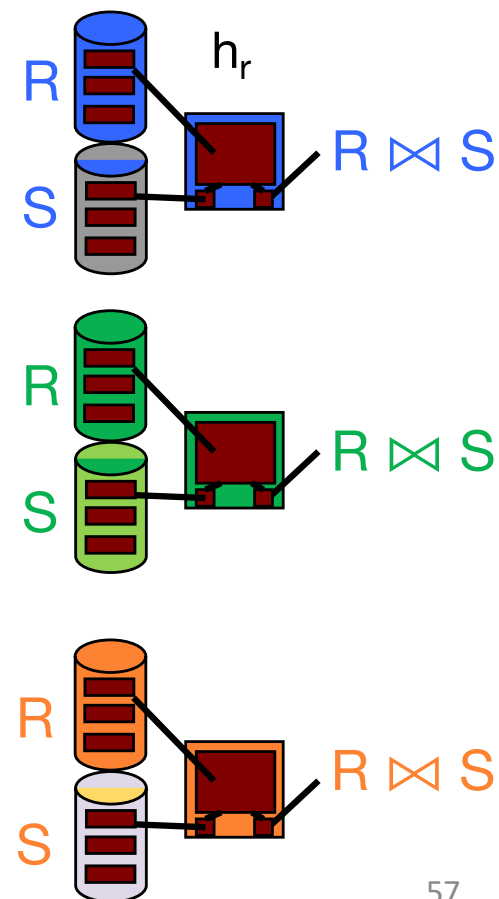
Parallel Grace hash join

- Pass 2 is local Grace hash join per node
 - Complete independence across nodes



Parallel Grace hash join

- Pass 2 is local Grace hash join per node
 - Complete independence across nodes in Pass 2



So easy!

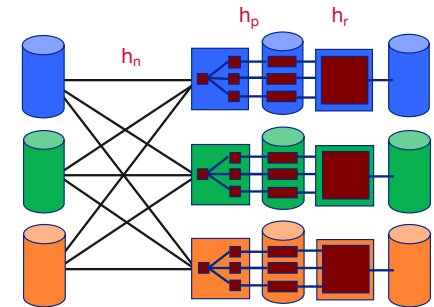
Nearly same as single-node Grace hash join

Near-perfect speed-up, scale-up!
Every component works at its top
speed, no waiting

Quick quizz

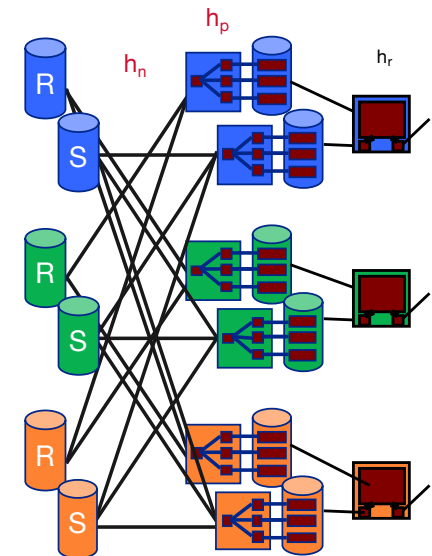
T/F: In parallel hashing...

- A. The main change over single-node is the hash-based shuffle at the beginning
- B. The first pass can do parallel scans
- C. Each node waits to consume the shuffle input before forming partitions
- D. Recursive partitioning can be handled exactly as in single-node hashing



T/F: In parallel Grace hash join...

- A. The main change over single-node is the hash-based shuffle in the final phase
- B. Two tuples that need to join *always* end up at the same machine
- C. If one key is extremely popular, then some machine will get an unfair amount of work

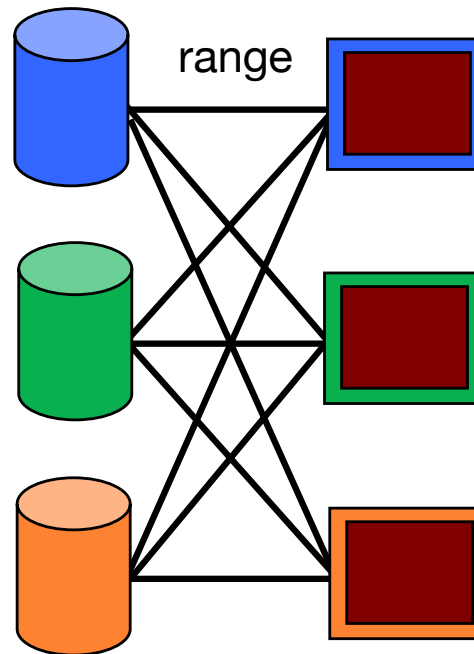


Sorting

INTRA-OPERATOR PARALLELISM

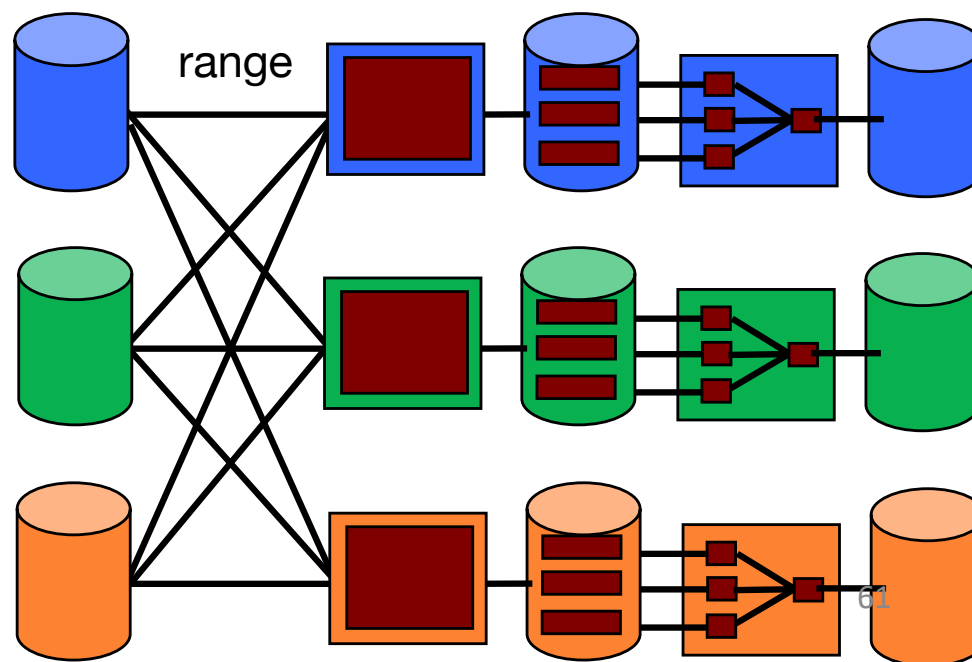
Parallelize me! Sorting

- Pass 0: shuffle data across machines
 - *streaming* out to network as it is scanned
 - which machine for this record?
Split on value range (e.g. $[-\infty, 10]$, $[11, 100]$, $[101, \infty]$).



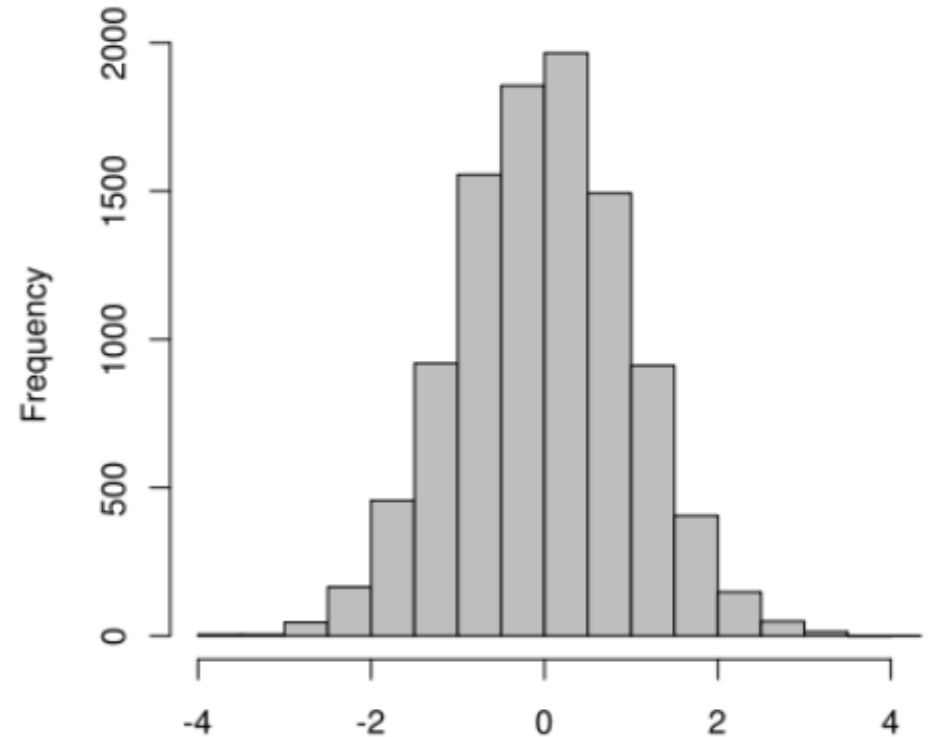
Parallelize me! Sorting

- Pass 0: shuffle data across machines
 - *streaming* out to network as it is scanned
 - which machine for this record?
Split on value range (e.g. $[-\infty, 10]$, $[11, 100]$, $[101, \infty]$).
- Receivers proceed with pass 0 *as the data streams in*
- Passes 1–n done independently as in single-node sorting
- *A wrinkle: How to ensure ranges are the same #pages?!*
 - *i.e. avoid data skew?*



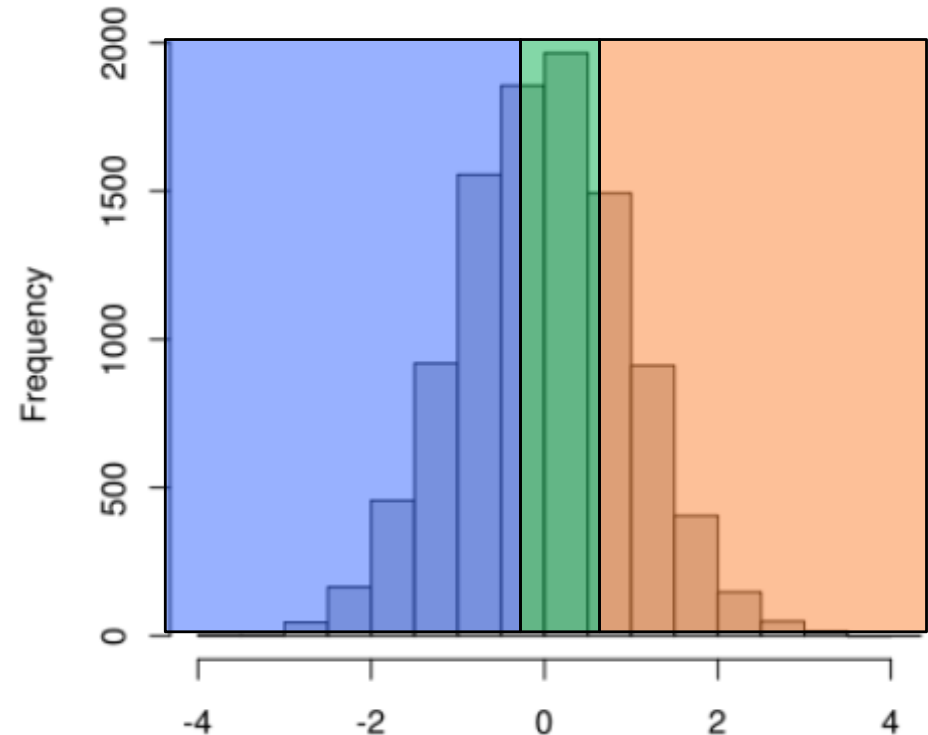
Range partitioning

- Goal: equal frequency per machine
- Note: ranges often don't divide x axis evenly
- How to choose?



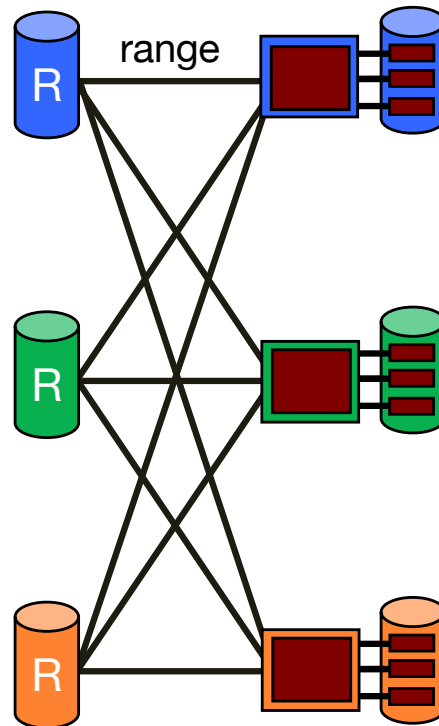
Range partitioning

- Goal: equal frequency per machine
- Note: ranges often don't divide x axis evenly
- How to choose?
 - Would be easy if data small
 - In general, can **sample** the input relation prior to shuffling, pick splits based on sample
 - Note: random sampling can be tricky to implement in a query *pipeline*; simpler if you materialize first.



Parallel sort-merge join

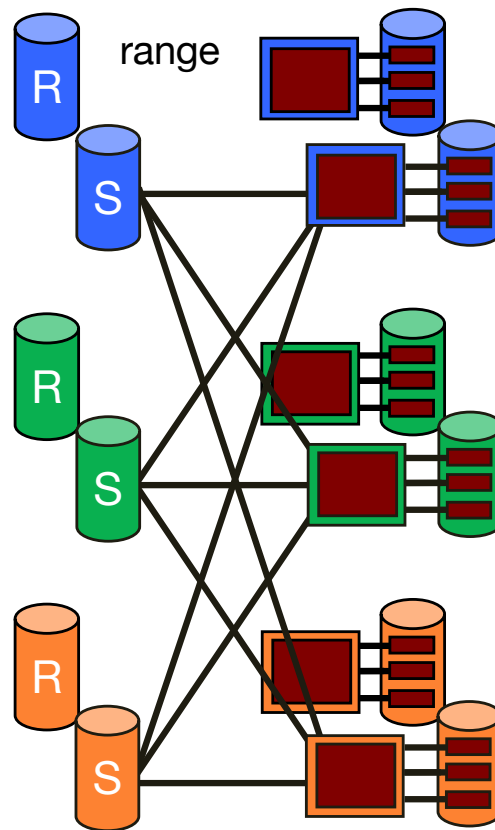
- Pass 0 .. n-1 are like parallel sorting above



Note: this picture is a 1-pass sort ($n=1$); this is pass 0

Parallel sort-merge join

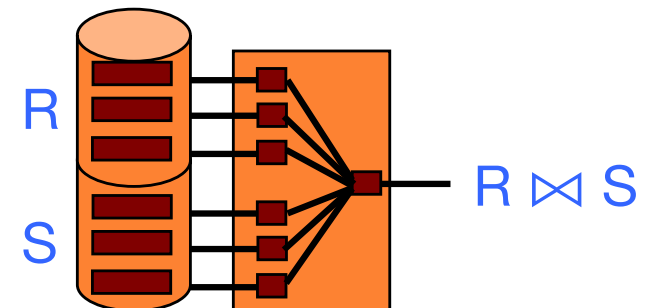
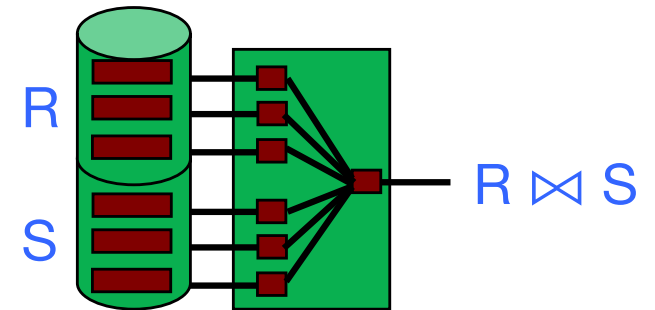
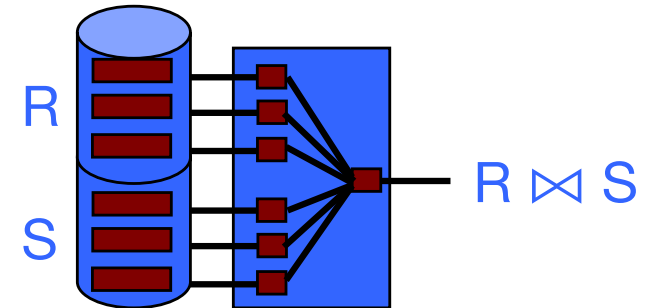
- Pass 0 .. n-1 are like parallel sorting above
 - But do it 2x: once for each relation, with same ranges



Note: this picture is a 1-pass sort ($n=1$); this is pass 0

Pass n (with optimization)

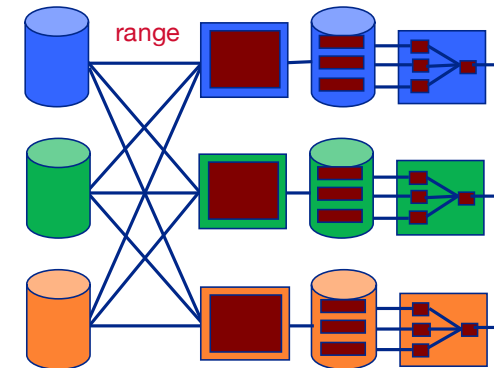
- Pass 0 .. n-1 are like parallel sorting above
 - But do it 2x: once for each relation, with same ranges
- Pass n: merge join partitions locally on each node



Quick quizz

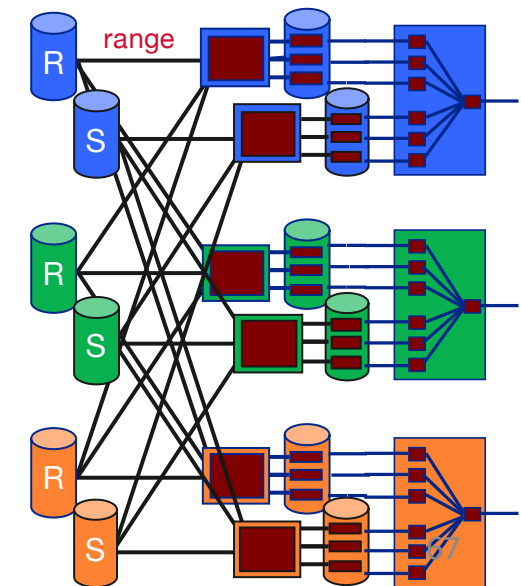
T/F: In parallel sorting...

- A. The main change over single-node is the range-based shuffle at the beginning
- B. The first pass can do parallel scans
- C. Each node waits to consume the shuffle input before forming partitions
- D. Multiple merge passes can be handled exactly as in single-node sorting



T/F: In parallel sort-merge join...

- A. The main change over single-node is the range-based shuffle in the final phase
- B. Two tuples that need to join *always* end up at the same machine
- C. If one key is quite popular, the range partitioning will adjust to balance load



Group By / Aggregate

INTRA-OPERATOR PARALLELISM

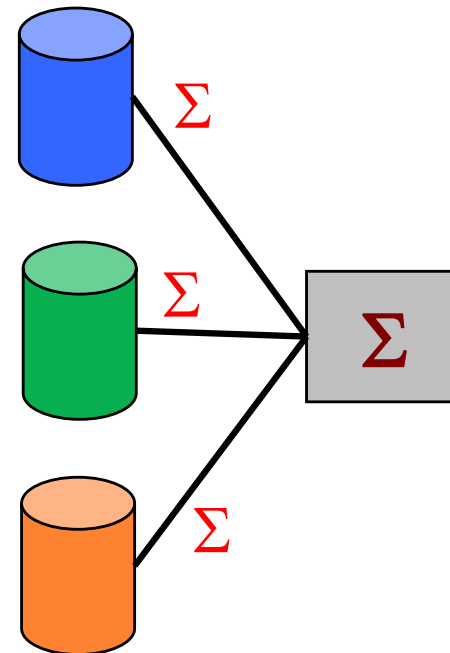
Parallel aggregates / group by

- Hierarchical **aggregation**

For each aggregate function, need a **global/local** decomposition:

- $\text{sum}(S) = \Sigma \Sigma (s)$
- $\text{count} = \Sigma \text{count} (s)$
- $\text{avg}(S) = (\Sigma \Sigma (s)) / \Sigma \text{count} (s)$
- etc...

```
SELECT SUM(sales)
FROM all_sales
```



Can you think of a summary statistic this doesn't work for?

Parallel aggregates / group by

- Hierarchical **aggregation**

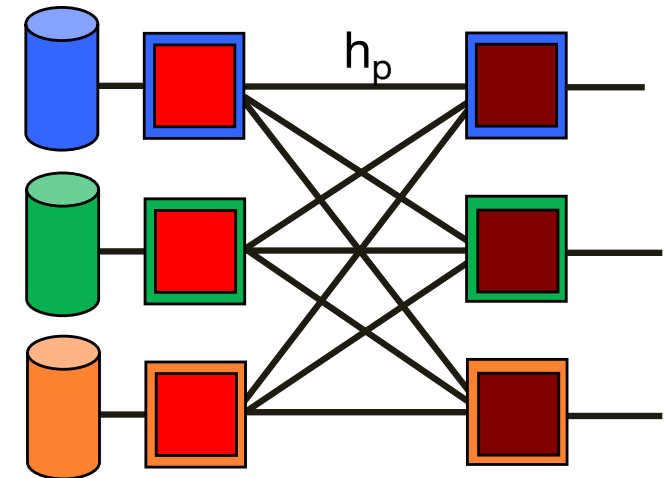
For each aggregate function, need a **global / local** decomposition:

- $\text{sum}(S) = \sum \sum (s)$
- $\text{count} = \sum \text{count}(s)$
- $\text{avg}(S) = (\sum \sum (s)) / \sum \text{count}(s)$
- etc...

- Naïve hash **group by**

- Local aggregation: in hash table keyed by group key k_i keep local agg;
 - In example, k is dno , agg is $(\text{sum}(\text{sales}), \text{count}(*))$
- Shuffle local aggs by a hash function $h_p(k_i)$
- Compute global aggs for each key k_i

```
SELECT dno, AVG(sales)
FROM all_sales
GROUP BY dno
```



Quick quizz

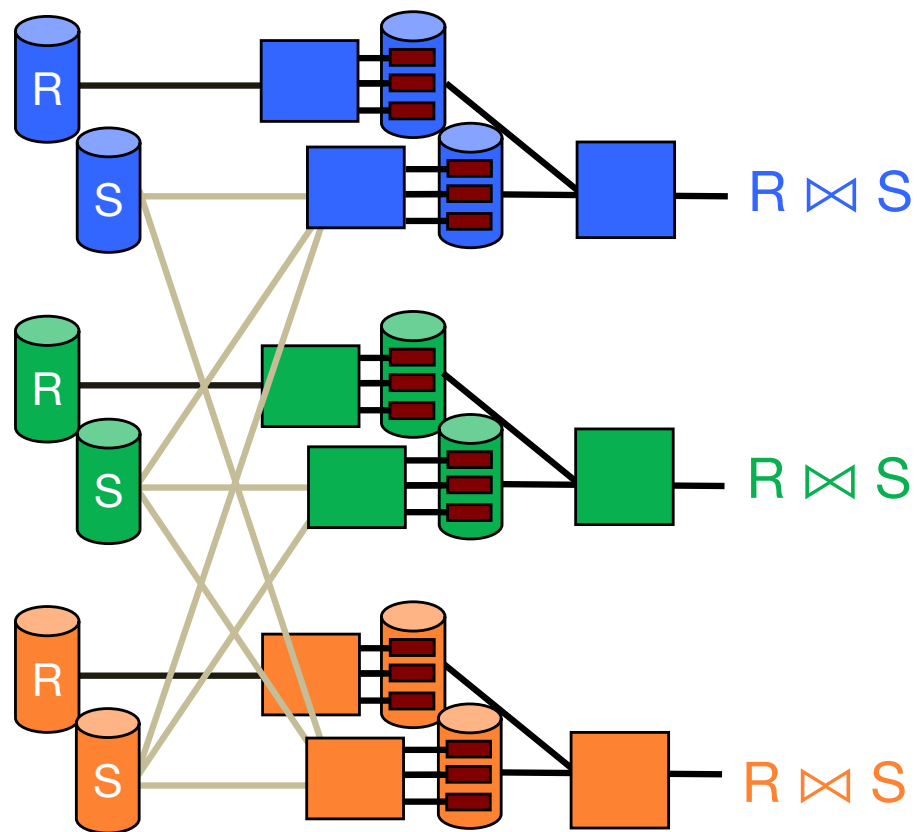
- Which of the following are true:
 - A. For a single group (i.e. no GROUP BY), there is no use for hierarchical aggregation
 - B. For GROUP BY, all local aggregates for a given group need to be shuffled to the same machine
- To do GROUP BY in a single pass on N rows
 - A. You need enough memory (across all machines) to hold all N rows
 - B. You need enough memory on each machine to hold (k_i, agg_i) for all distinct group values k_i
 - C. You need $O(\sqrt{N})$ memory across all machines

Joins: bigger picture

Alternatives:

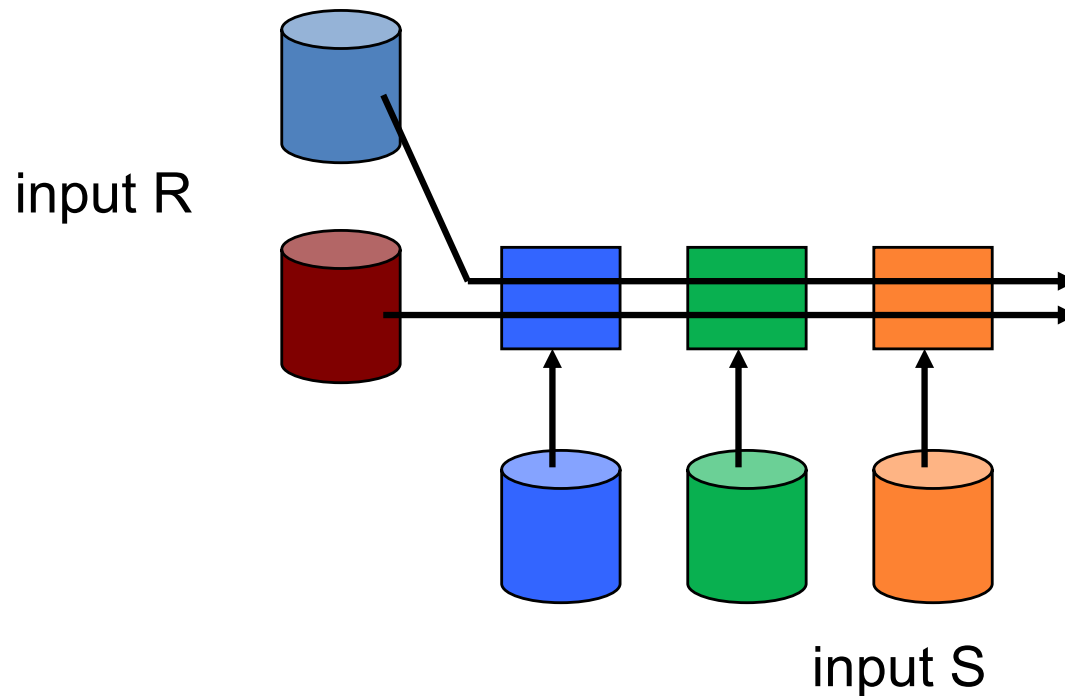
- symmetric shuffle
 - what we did so far
- asymmetric shuffle
- broadcast join

Join: asymmetric shuffle



If R already suitably partitioned, just partition S, then run local join at every node and union results.

“Broadcast” join



If R is small, send it to all nodes that have a partition of S .

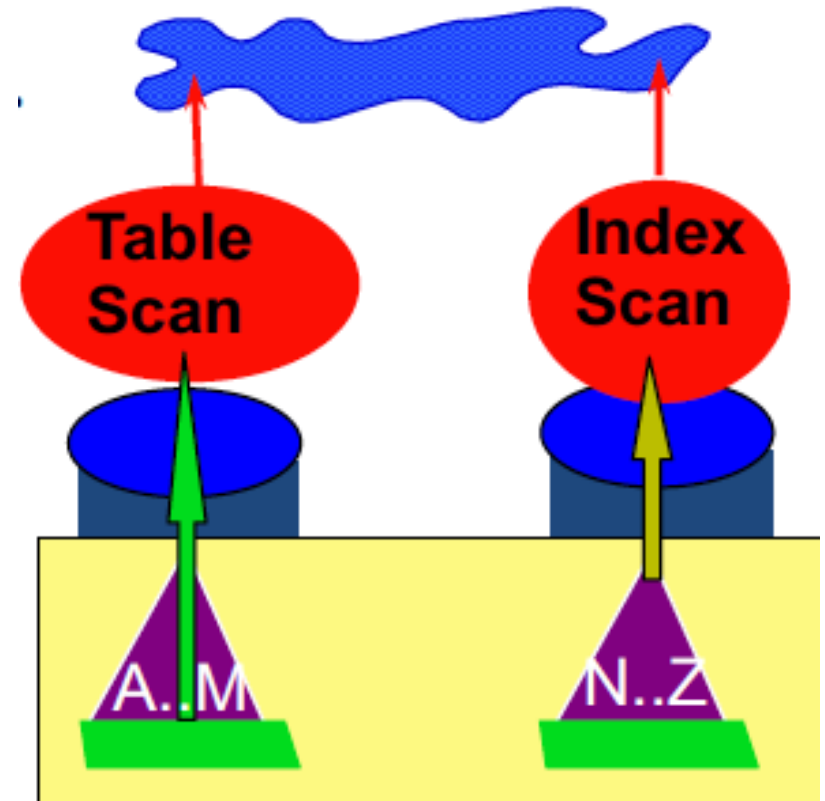
Do a local join at each node (using any algorithm) and union results.

Quick quizz

- Broadcast join is useful when one relation is
 - A. Very small
 - B. Large, but highly skewed
 - C. Partitioned on the join key
- Asymmetric shuffle is useful when one relation is:
 - A. Very small
 - B. Large but highly skewed
 - C. Partitioned on the join key

Quick quizz

- Can the best serial plan not correspond to the best parallel one ?
- Yes ! Example:
 - Table partitioned with local secondary index at two nodes
 - Range query: all of node 1 and 1% of node 2
 - Node 1 should do a full scan of its partition
 - Node 2 should use the index

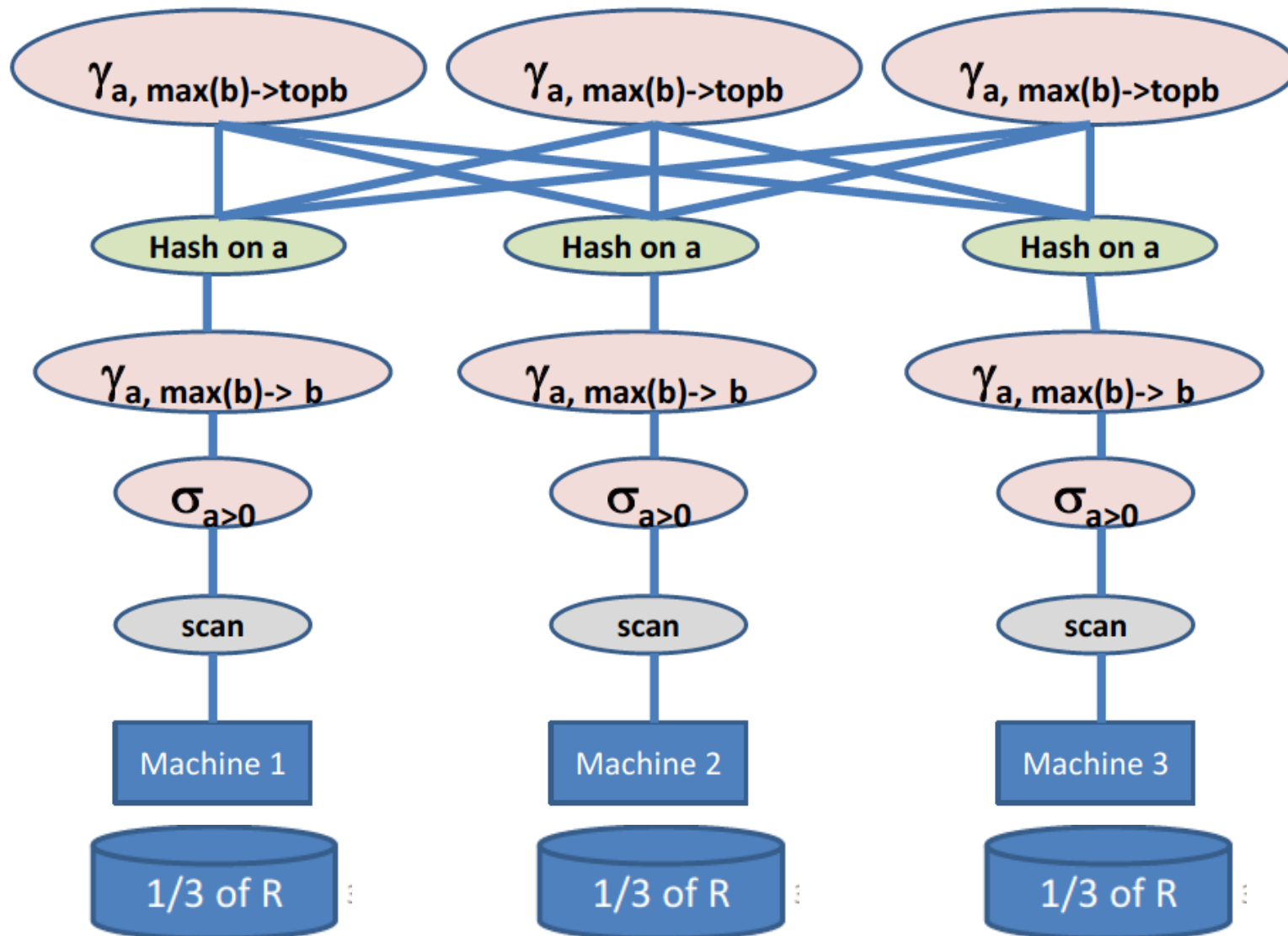


Quick quizz – Execution plan

```
SELECT a, max(b) as topb  
FROM R  
WHERE a > 0  
GROUP BY a
```

- $R(a,b)$ is horizontally partitioned across $N = 3$ machines
- Each machine locally stores approximately $1/3$ of the tuples in R
- Tuples are randomly organized across machines (i.e., R is block-partitioned across machines)
- Give a RA plan for this query and show how it will be executed across the 3 machines

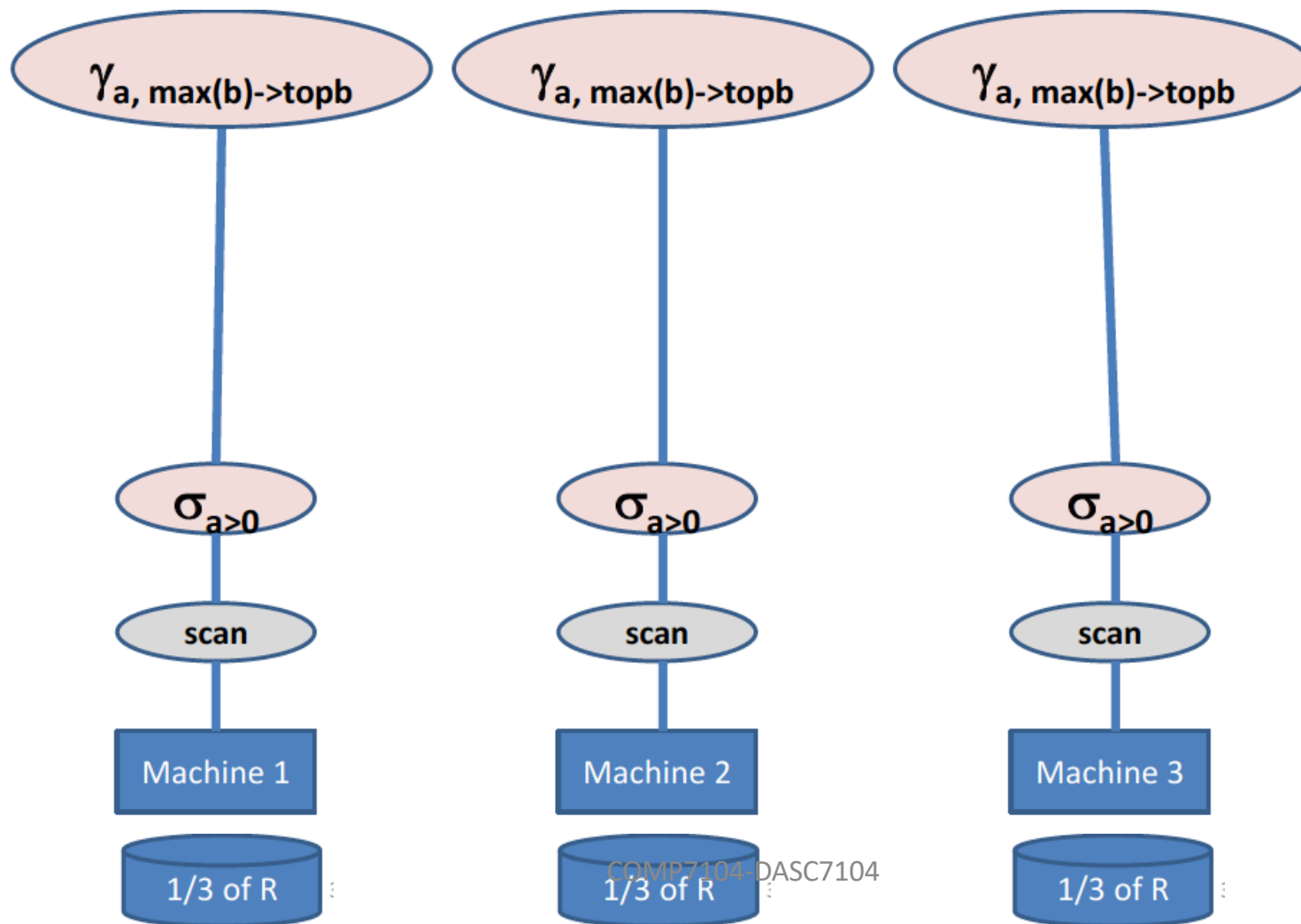
Quick quizz – Execution plan



Quick quizz – Execution plan

What would change if we first hash partitioned R on A, before executing the query ?

It would avoid the data reshuffling phase, would compute the aggregates locally



Parallel DBMS summary

- Parallelism natural to query processing:
 - Both pipeline and partition
- Shared-nothing vs. shared-memory vs. shared disk
 - Shared-memory easiest SW, costliest HW
 - Doesn't scale, not common
 - **Shared-nothing**
 - Cheap, scales compute well
 - Harder to implement, harder to scale capacity, guarantee consistency
 - **Shared disk**
 - Compute and storage can scale independently, most cloud-native systems
 - Pbs. related to concurrency control
- Intra-operator, inter-operator, & inter-query parallelism all possible

Parallel DBMS summary (continued)

- Data layout choices important!
- Most DB operations can be done partition-parallel
 - Sort
 - Sort-merge join, grace hash-join
- Complex plans
 - Allow for pipeline parallelism, but sorting, hashing may **block** the pipeline
 - Partition / pipeline parallelism mixed via bushy trees